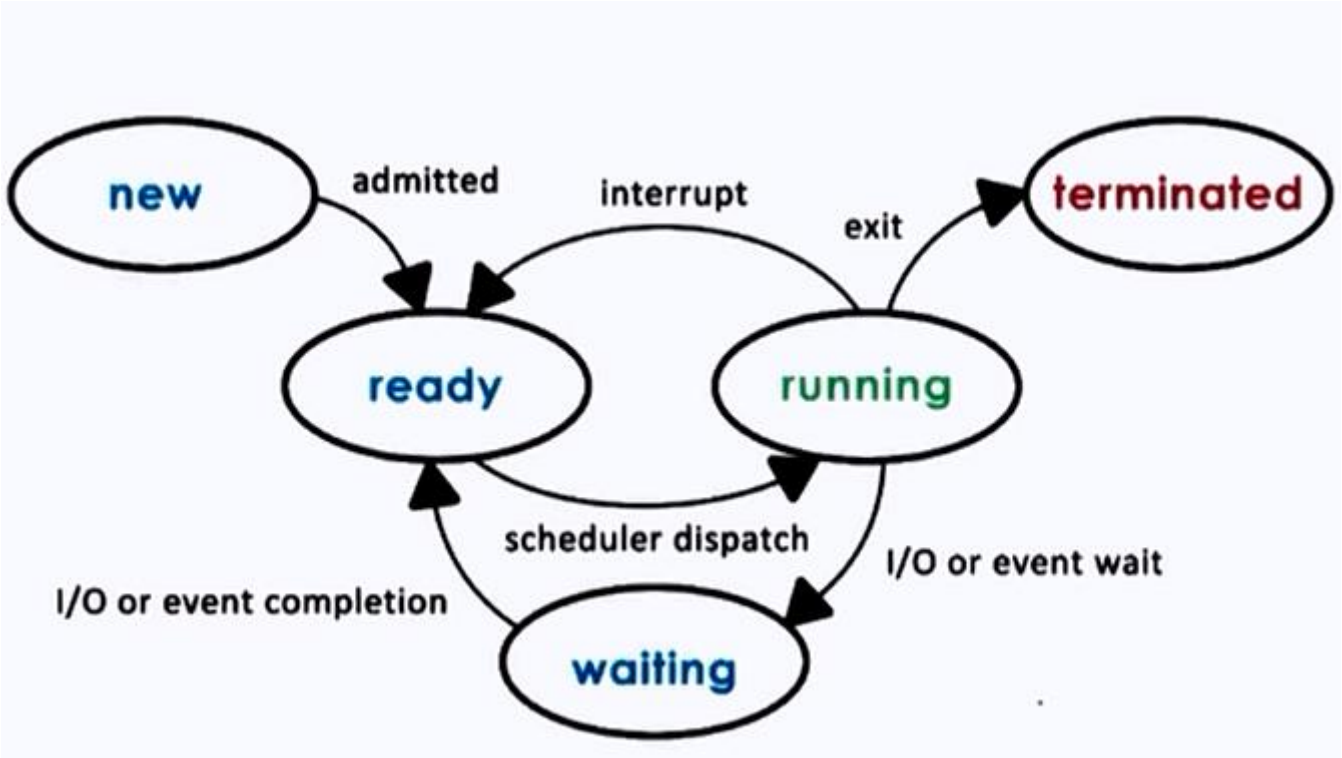
İşletim Sistemleri
(Operating Systems)



Process (İşlem-Süreç)

İşletim Sistemi Process Yönetimi

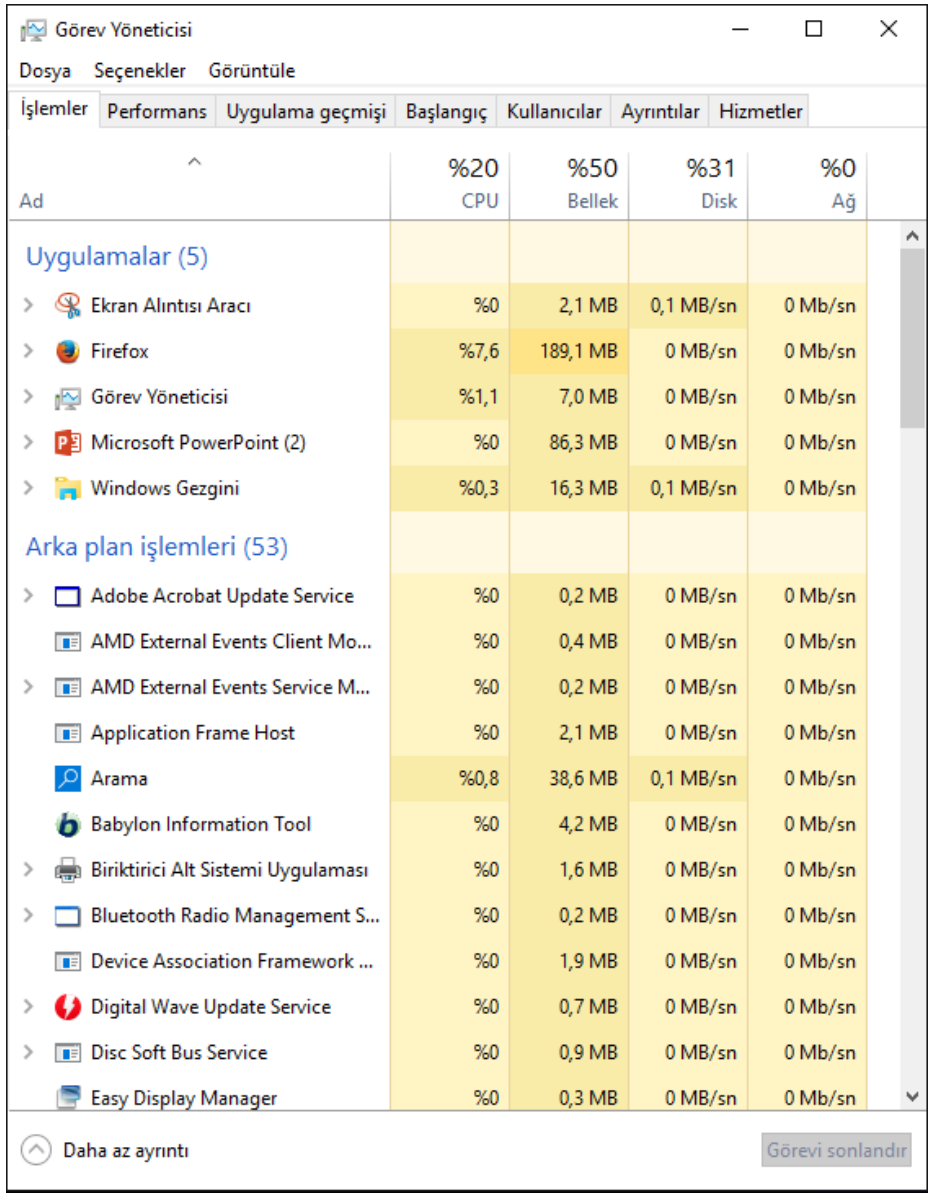
- Process oluşturma ve sonlandırma
  (Process creation and termination)
- Process değiştirme
  (Process switching)
- Process zamanlama
  (Process scheduling)
- Process senkronizasyonu ve process'ler arası iletişim
  (Process synchronization and inter-process communication)

Process Tanımı

- **Process:** Bir işletim sistemi üzerinde belleğe yüklenerek çalıştırılan program parçalarına verilen isimdir. (komut satırından yada mouse+click ile)
- Bir program pasif varlıktır (passive entity), bir process ise aktif varlıktır (active entity).
- Bir bilgisayar programı durgun bir komutlar dizisi şeklinde bulunurken, bir process bu komutlar dizisinin işletilmesi durumudur.
- Process'lerin bazısı işletim sisteminin çalıştırılmasıyla ilgilidir (sistem process'leri), diğerleri esas olarak kullanıcı programlarının yürütülmesini gerçekleştirir (kullanıcı process'leri).

- *Program: Verileri işlemek üzere oluşturulan komutlar/komut kümeleridir.*
- *Process: Bir programın bilgisayar sistemlerince işletimi esnasında aldığı isimdir.*

Process Tanımı



Process Tanımı

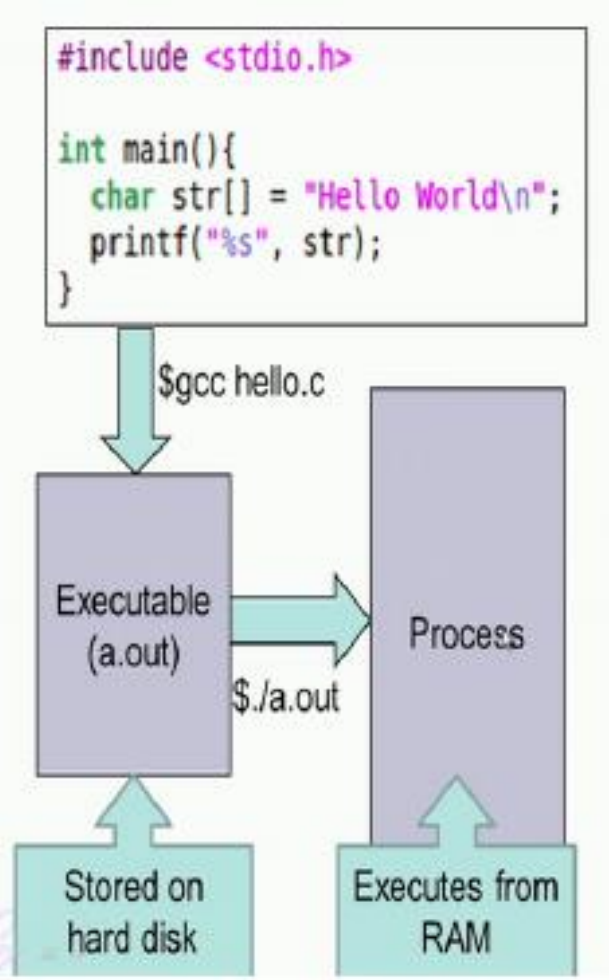
- Klasik process; Von Neumann mimarisi bilgisayarlarda işlemcide yürütülmekte olan programa ait bir tabirdir.
- Tanenbaum'a göre, bir process çalışan bir programın bir soyutlamasıdır.
- Her program için en az bir process tanımlanır.
- Aynı programa ilişkin birden fazla process olabilir.
- Her process, yeni başka process’ler de üretebilir.
- Process’ler tipik olarak birbirinden bağımsızdır.

Process Tanımı

C'de yazılan bir programı düşünün. Bu program ekrana "Hello World" yazıyor. Bu programı derlemek ve çalıştırmak için öncelikle gcc gibi bir derleyiciyi kullanabiliriz.

hello.c olarak yazılmış program kodu derlendikten sonra a.out gibi bir isimlendirme ile yürütülebilir dosya olarak sabit diskte saklandığını düşünelim.

a.out programı çalıştırıldığında bellekde bir process olarak oluşturulur. Yani, bu process esasen bellekde yürütme sürecindeki bir programdır.



Process Tanımı

- Bir process yapması gereken işi tamamlamak için, bellek, işlemci süresi, ve giriş/çıkış cihazları gibi kaynaklara ihtiyaç duyar.

- Process yürütüldüğü sürece bu kaynaklardan ihtiyaç duyduğunu kullanır, çalışması sonlanınca işletim sistemi process'e tahsis edilmiş kaynakları serbest bırakır.

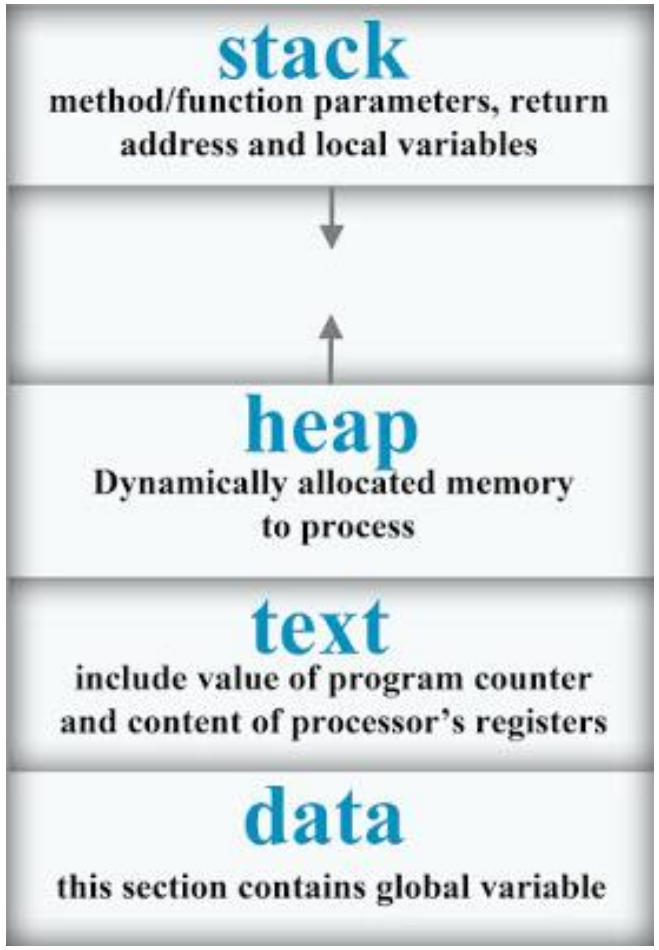
- Çok görevlilik (Multitasking) özelliğine sahip olan işletim sistemleri, aynı anda birçok process'in yürütülmesine izin verir.

- Tek çekirdekli işlemciye sahip olan bilgisayarlarda bu iş zaman paylaşımı esasına göre yapılır.

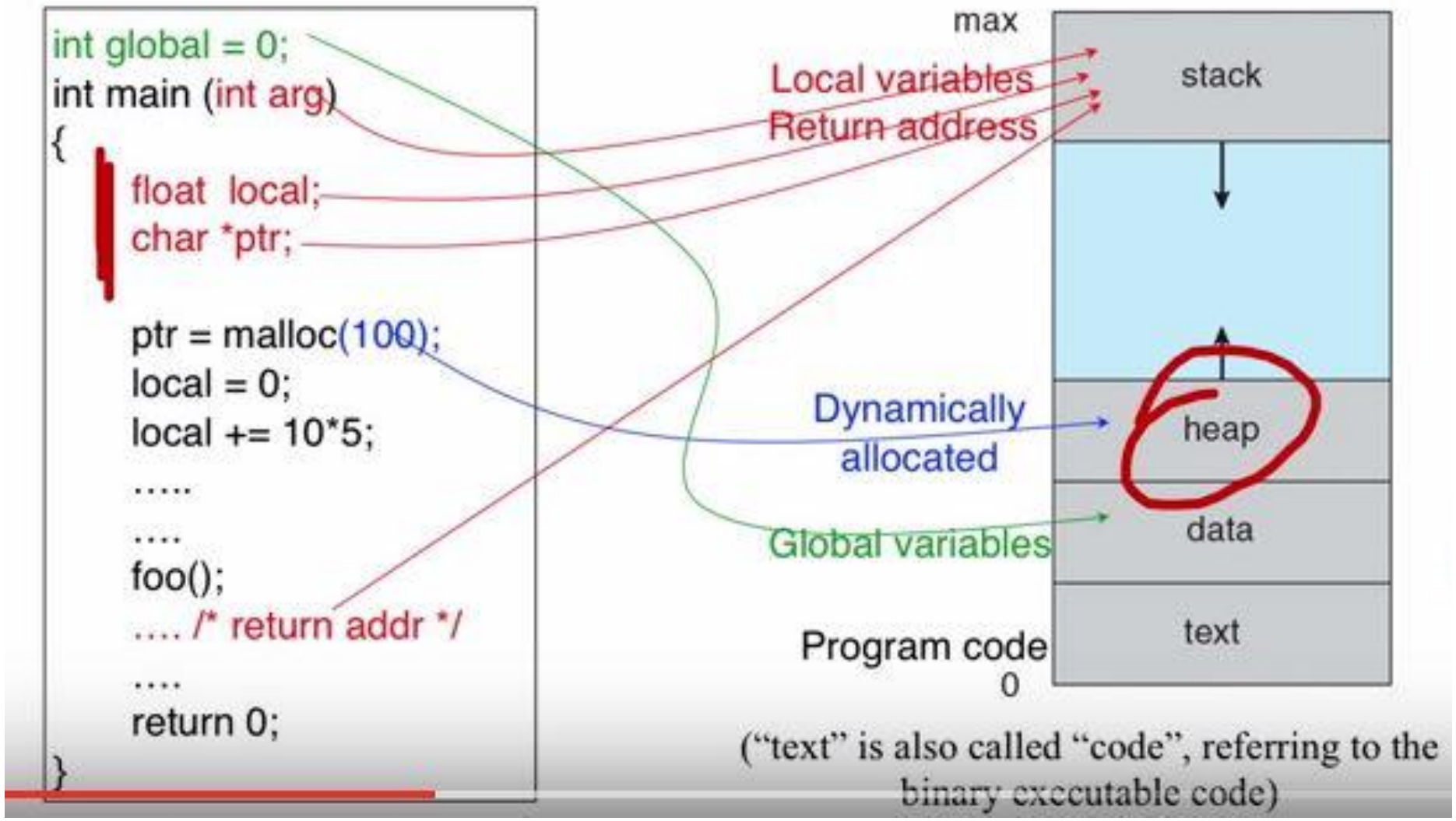
Process Bellek Modeli

- **Process bellek modeli bir işlemciden diğerine değişmekle birlikte genel olarak bir modele sahiptir.**
- **Text (Code):** Derlenmiş program kodunu ve fonksiyonları saklar. Salt okunur bir bölümdür çünkü kod yürütme sırasında değiştirilmez.
- **Data:** Global değişkenler ve fonksiyonlarda static olarak tanımlanan değişkenler saklanır. Data segmenti okuma ve yazma izinlerine sahiptir. Initialized Data Segment ve Uninitialized Data Segment olarak iki bölümü vardır.
- **Stack:** local değişkenleri, method/fonksiyon parametreleri ve return adreslerini saklar.
- **Heap:** Dinamik bellek ayırma için kullanılır.

- Stack ve Heap, process'in boş alanının zıt uçlarında başlar ve birbirine doğru büyürler.



Process Bellek Modeli

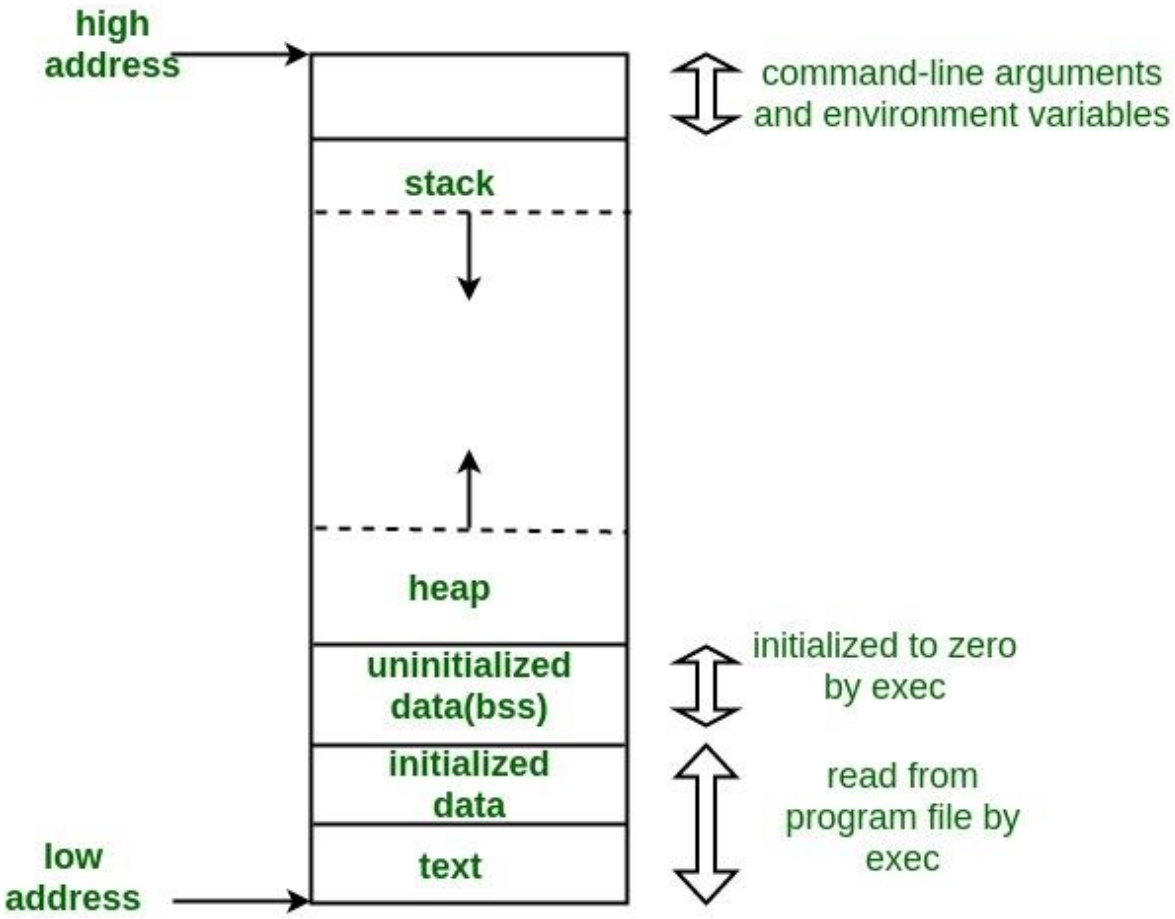


Data segment

Initialized Data Segment : Değişkenlerin değerleri program başlatılmadan önce atanmıştır.
Uninitialized Data Segment : Genellikle “bss” segmenti olarak adlandırılır. Bu bölümdeki global ve statik değişkenler, program başlatılmadan önce çekirdek (kernel) tarafından aritmetik 0‘da başlatılır.

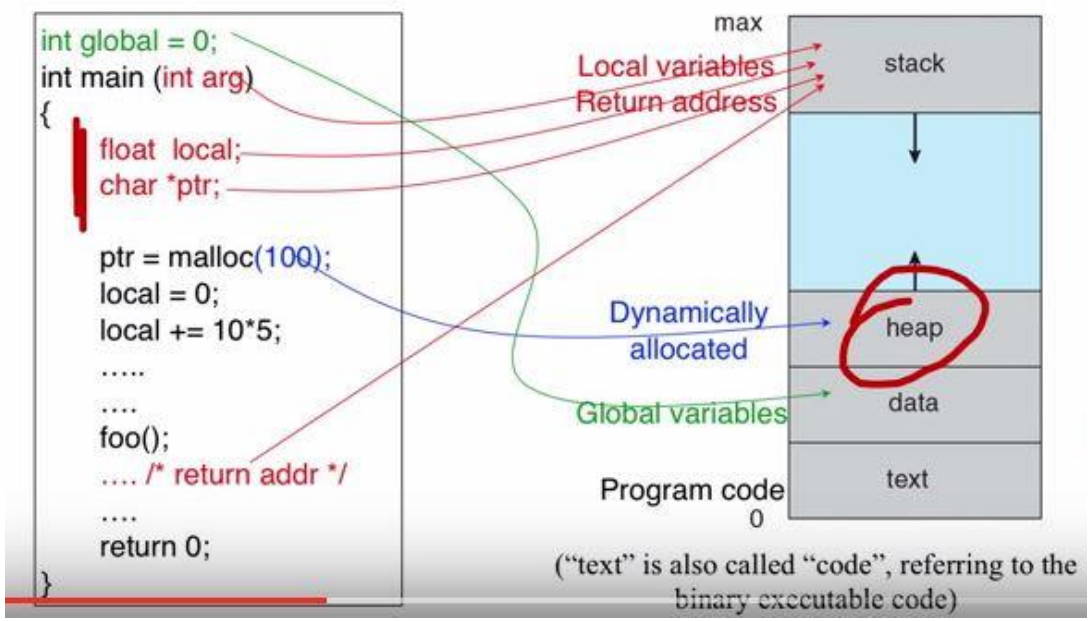
initialized data :
int i = 10
static int i = 10

uninitialized data :
int i
static int i



Stack Segment

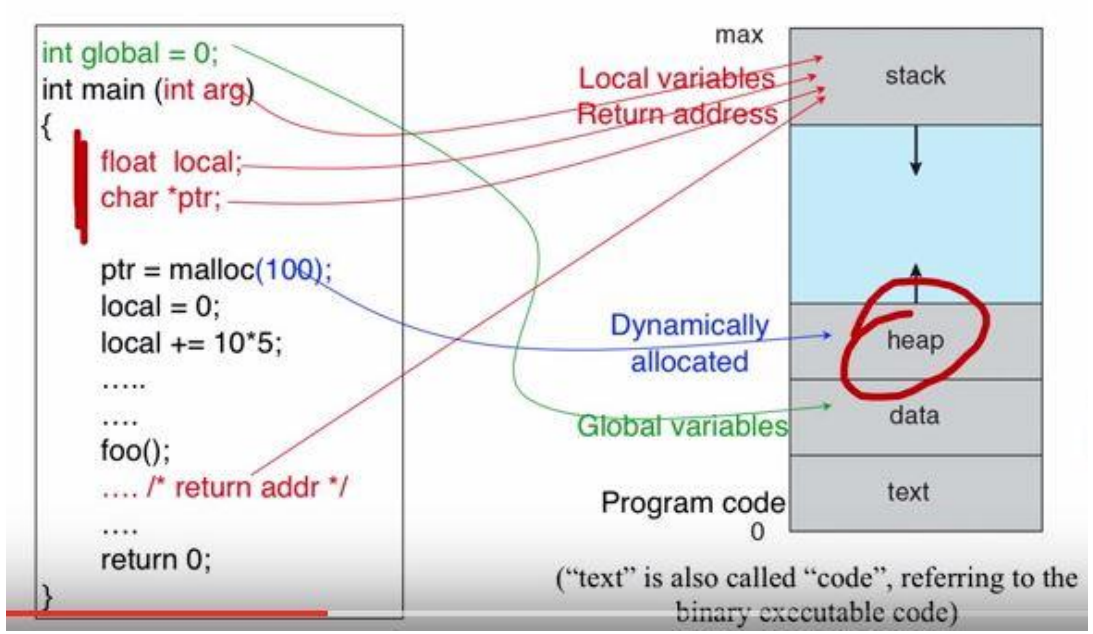
- **Stack (Yığın) :** Yığın, çalışma zamanında belleğin değişkenler için dinamik olarak ayrılabileceği bir alandır. Yerel değişkenler, method/fonksiyon parametreleri, return adresi ve değeri gibi değer tipleri, pointer ve adresler saklanır.
- Yığının çalışması otomatik olarak gerçekleşir.
- Yerel değişkenler erimi başladığı yerde otomatik olarak yığında oluşturulur, erimi dışına çıkıldığında ise yine otomatik olarak yok edilir. Bu yüzden bu tür değişkenler bazen geçici ya da otomatik değişken olarak da adlandırılır.
- Yığın'da veriler üst üste (LIFO) mantığında dizilir ve sırası gelmeden aradaki bir değer ile işlem yapılamaz.



Bir fonksiyon çağrıldığında, yığının bir bölümü fonksiyon ile ilgili bilgileri (yerel değişkenleri, dönüş adresi vb.) depolamak için tahsis edilir. Bu bölüme yığın çerçevesi (stack frame) denir. Yığın segmentinde her fonksiyonun kendi yığın çerçevesi vardır.

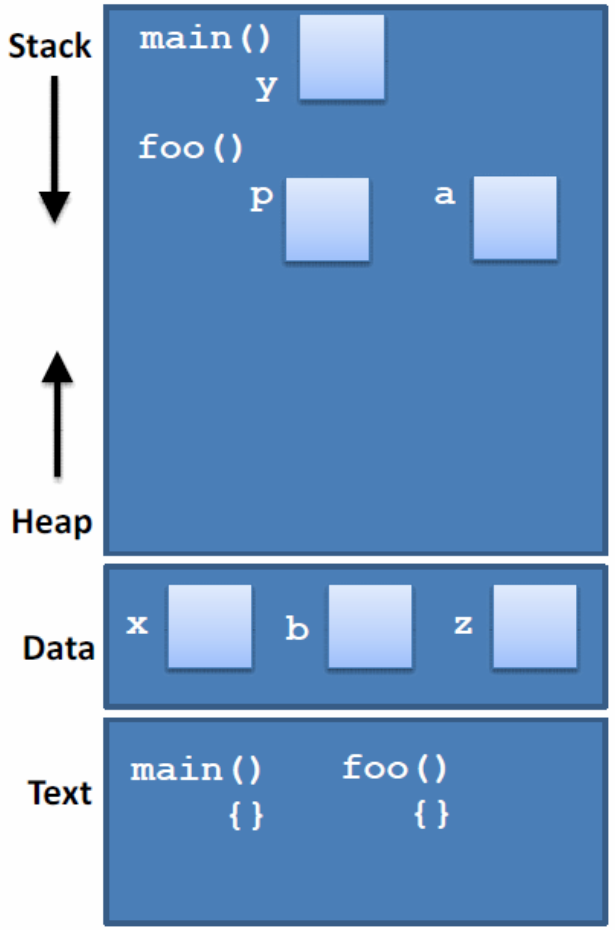
Heap Segment

- **Heap :** Program çalışması esnasında boyutu değişen değer kullanımında işe yarar.
- Heap alanının yönetiminden programcı sorumludur.
- Programcı, C'de malloc() ve free() fonksiyonlarını, C++'da ise new ve delete operatörlerini kullanarak ihtiyacı kadar alanı alır ve kullanımı bitince aldığı alanı geri vermekle sorumludur.
- Heap'da veriler karışık bir şekilde saklanır, bu yüzden Heap'deki bir veriye erişmek Stack'daki bir veriye erişmeye göre daha maliyetli bir işlemdir.
- Heap alanı, bir process'deki tüm paylaşılan kütüphaneler ve dinamik olarak yüklenen modüller tarafından paylaşılır.

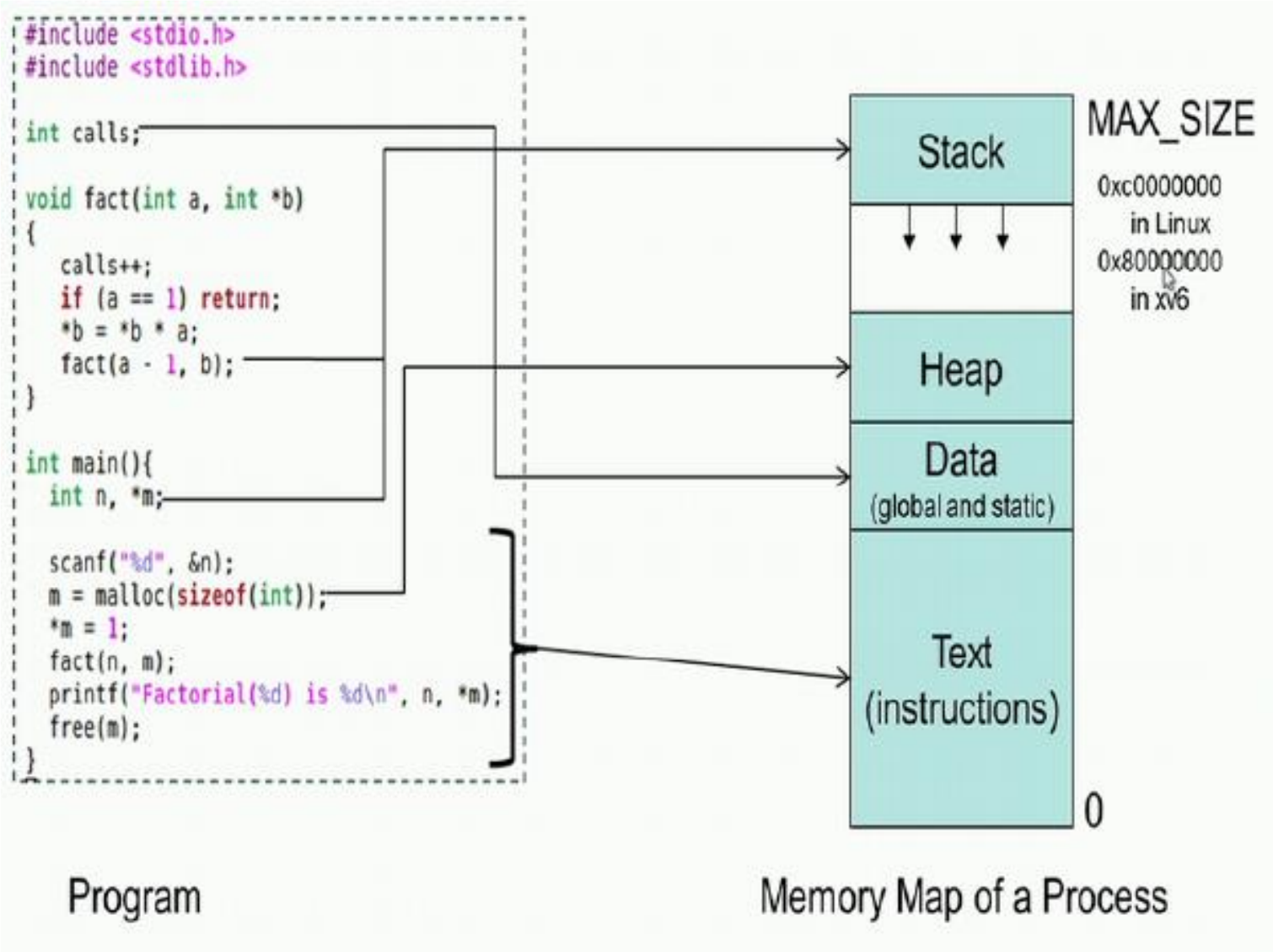


Process Bellek Modeli

```
int x;
static int b = 4;

main() {
  int y;
  static int z;

  y = b;
  z = foo(y);
}

foo( int p ) {
  int a;
  a = p + 2;
  return(a);
}
```



Process Bellek Modeli



Örnek

1. Aşağıdaki basit C programını kontrol edin.

```
#include <stdio.h>
int main(void)
{
   return 0;
}
```

```
[CentOS]$ gcc memory-layout.c -o memory-layout
[CentOS]$ size memory-layout
text      data      bss      dec      hex    filename
1115      552       8        1675     68b    memory-layout
```

text, data ve bss bölümlerinin boyutları (bayt cinsinden).

Örnek

2. Programa bir global değişken eklendiğinde,

```
#include <stdio.h>
int global; /* Uninitialized variable stored in bss*/
int main(void)
{
   return 0;
}
```

```
[CentOS]$ gcc memory-layout.c -o memory-layout
[CentOS]$ size memory-layout
```

text	**data**	**bss**	dec	hex	filename
1115	552	8	1675	68b	memory-layout

Örnek

3. bss'de saklanan bir statik değişken eklendiğinde.

```
#include <stdio.h>
int global; /* Uninitialized variable stored in bss*/
int main(void)
{
   static int i; /* Uninitialized static variable stored in bss */
   return 0;
}
```

```
[CentOS]$ gcc memory-layout.c -o memory-layout
[CentOS]$ size memory-layout
text      data      bss      dec      hex   filename
 1115      552      16      1683      693   memory-layout
```

Örnek

4. Şimdi Data Segmentinde (DS) saklanacak statik değişkeni başlatalım.

```
#include <stdio.h>
int global; /* Uninitialized variable stored in bss*/
int main(void)
{
   static int i = 100; /* Initialized static variable stored in Data Segment*/
   return 0;
}
```

```
[CentOS]$ gcc memory-layout.c -o memory-layout
[CentOS]$ size memory-layout
```

text	**data**	**bss**	dec	hex	filename
1115	556	12	1683	693	memory-layout

Örnek

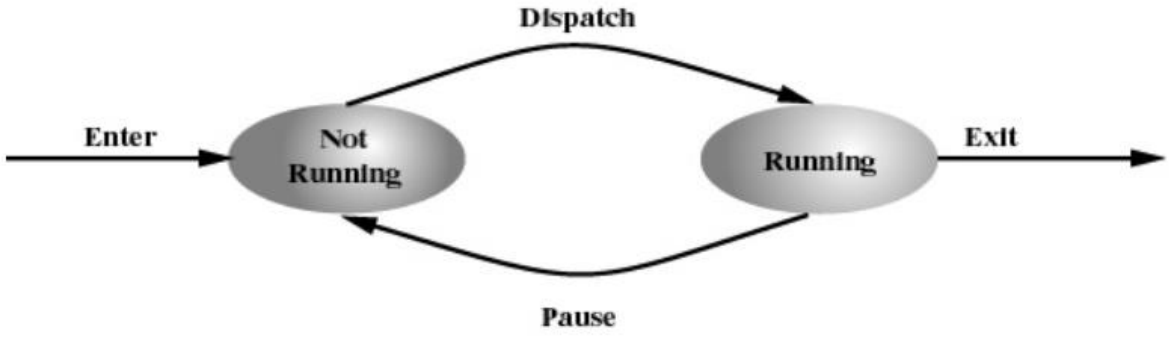
5. Daha sonra Data Segmentine (DS) kaydedilecek olan global değişkeni başlatalım.

```
#include <stdio.h>
int global = 10; /* initialized global variable stored in Data Segment*/
int main(void)
{
   static int i = 100; /* Initialized static variable stored in Data Segment*/
   return 0;
}
```

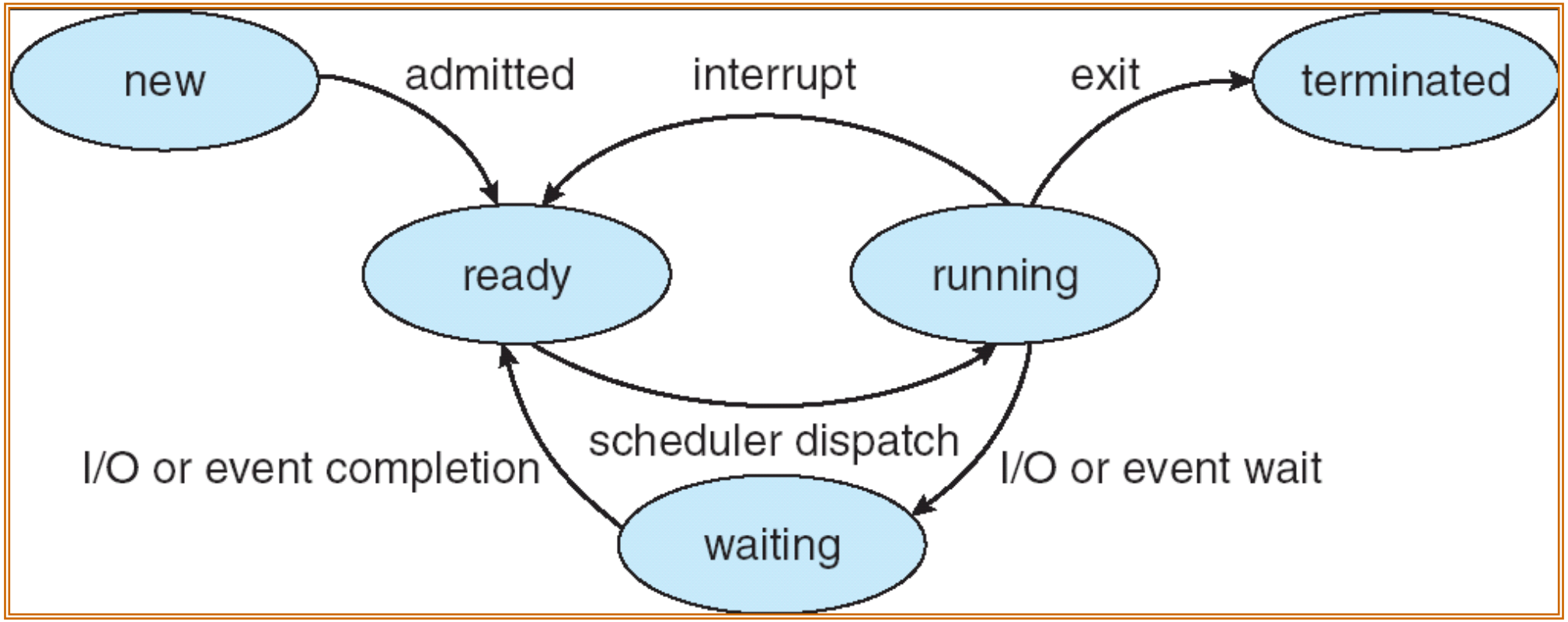
```
[CentOS]$ gcc memory-layout.c -o memory-layout
[CentOS]$ size memory-layout
text      data      bss      dec      hex   filename
1115      560       8        1683     693   memory-layout
```

İki Durumlu Process Modeli

- Herhangi bir zaman diliminde bir process CPU tarafından ya işletiliyordur yada işletilmiyordur.
- Yani process ya Koşuyor (Running) yada Koşmuyor (Not Running) durumdadır.



Beş Durumlu Process Modeli



Process Durumları

- Bir process'in herhangi bir andaki eylemi, process'in o andaki durumunu belirler. Bir process 5 durumda olabilir.

1. Yeni (New) : Process'in oluşturulduğunu gösterir.

2. Hazır (Ready) : Bu durumda bulunan bir process CPU'da koşturulmaya hazır durumdadır ve ana bellekte tutulmaktadır.

- Process, hazır kuyruğunda (Ready queue) sırasının gelmesini bekler. Hazır durumda birden fazla process olabilir.
- İşlemci zamanlama algoritmalarına (CPU scheduling algorithm) göre de sırası gelen process hazır kuyruğundan alınarak görevlendirici (dispatcher) tarafından CPU'ya gönderilir.

Process Durumları

3. Koşuyor (Running) : Process, işlemciyi şu anda kullanmaktadır.

4. Bekliyor/Bloke (Waiting/Blocked) : Bir process, başka bir process tarafından kullanılan bir kaynağı beklediğinde veya şu anda hazır olmayan bir kaynağı kullanmak ihtiyacı olduğunda bekleme durumuna geçer. (I/O or event wait)

5. Sonlanıyor (Terminated) : Process'in sonlandırıldığını gösterir.

Process Durumları

- **Admitted:** New durumundaki bir process'in Ready durumuna geçmesidir. (Created)

- **Scheduler Dispatch:** Ready Durumundaki process'in Running durumuna geçmesidir. (Dispatched)
- Dispatcher, işlemci zamanlama algoritmasına (CPU scheduling algorithm) göre hazır kuyruğunda (ready queue) beklemekte olan process'den sıradakini alıp işlemciye yollayan programın ismidir.

- **Interrupt:** Running durumundaki Process'in Ready durumuna geçmesidir. (run-time expires)

- **I/O or Event Wait:** Process bir eylemin gerçekleşmesini beklediği durumda running'den, bekleme/bloke durumuna geçer. (Blocks)

- **I/O or Event Completion:** Waiting durumundaki process'in beklediği durum gerçekleştiğinde Ready durumuna geçer. (event accours)

- **Exit:** Çalışması biten processin running durumundan Terminated durumuna geçmesidir. (finished)

Process Durum Geçişleri

- Bir process'in durumlar arasında alabileceği altı geçiş vardır:
- **1. Create -> Ready:** Process oluşturuldu ve çalışmaya hazır
- **2. Ready -> Running:** Zamanlayıcı (Scheduler), process'i CPU'da koşturmaya karar verir
- **3. Running -> Ready:** Zamanlayıcı, koşturulmakta olan process'i durdurmaya ve bunun yerine başka bir process'i koşturmaya karar verir.
- **4. Running -> Waiting/Blocked:** Process'in koşturmaya devam etmeden önce; kaynak, veri, G/Ç, olay vs. ihtiyacı vardır.
- **5. Running -> Exit:** Process sonlandırılır.
- **6. Waiting/Blocked -> Ready:** Process'in beklediği kaynak, veri, G/Ç, olay vs. gerçekleşmiştir.

Process Durumları

- Hazır (Ready) durumda bulunan tüm process'ler Hazır durum kuyruğunda bekletilirler.
- Görevlendirici (dispatcher), Hazır durumda olan bir process'i işlemci zamanlama algoritma stratejisi çerçevesinde seçerek çalışması için işlemciye (CPU) gönderir. Böylece process durumu Hazır (Ready) durumundan Koşuyor (Running) durumuna dönüşmüş olur.
- Process, hazır olmayan bir kaynak beklediği zaman, (giriş/çıkış işlemi veya mesaj alma vb. durumlarda) Hazır (Ready) durumdan Bekleme/Bloke (Waiting/Blocked) durumuna geçer.

Process Durumları

- Bir process'in daha önceden talep ettiği kaynak işletim sistemi tarafından process'e tahsis edildiği zaman process Bekleme/Bloke durumdan Hazır duruma geçiş yapar ve tekrar hazır kuyruğuna girer.

- İşletim sistemi bir kesme (interrupt) vasıtası ile hali hazırda işlemciyi kullanan process'in koşturulmasını durdurabilir ve process tekrar Hazır kuyruğuna gönderilir.

Process Durumları

- Örnek olarak;
- Hesap makinası programı çalıştırılmak istendiğinde onu çalıştıran process önce **Hazır kuyruğuna geçer**, sonra işlemcide işlem görerek **Koşuyor durumuna geçer** ve program açılır.

- Daha sonrasında ise hesap makinesi programını çalıştıran process, **"Bekleme/Bloke durumu"na geçer**.

- Kullanıcıdan klavye vasıtasıyla rakam girişi beklenir. Rakam girişi veya bir işlem yapılmadığı sürece hep **Bekleme/Bloke** durumunda kalır.

Process Durumları

- Process çalışırken klavye, gibi dışsal bir olayı bekleyecek bir işlem başlatırsa process'in çalışıyor olma durumu sona erdirilir ve process bekleme/bloke olmuş duruma getirilir.
- Çok process'li çalışmada parçalı çalışma süresi (quantum) boyunca meşgul bir döngü içerisinde sürekli klavyenin yoklanması (polling) etkin bir yöntem değildir.
- Bu yüzden sistem çalışmakta olan process'i bekleme/bloke durumuna aldıktan sonra çalışmaya hazır olan process'lerden uygun olan bir başkasını seçerek onun çalışmasını sağlar.
- Bekleme/Bloke edilen process'ler bekleme kuyruklarına alınmaktadır.
- Dışsal olayı beklemek için bekleme/bloke durumundaki bir process, dışsal olay gerçekleştiğinde bekleme/bloke durumundan çıkartılarak çalışmaya hazır (ready) duruma getirilir.

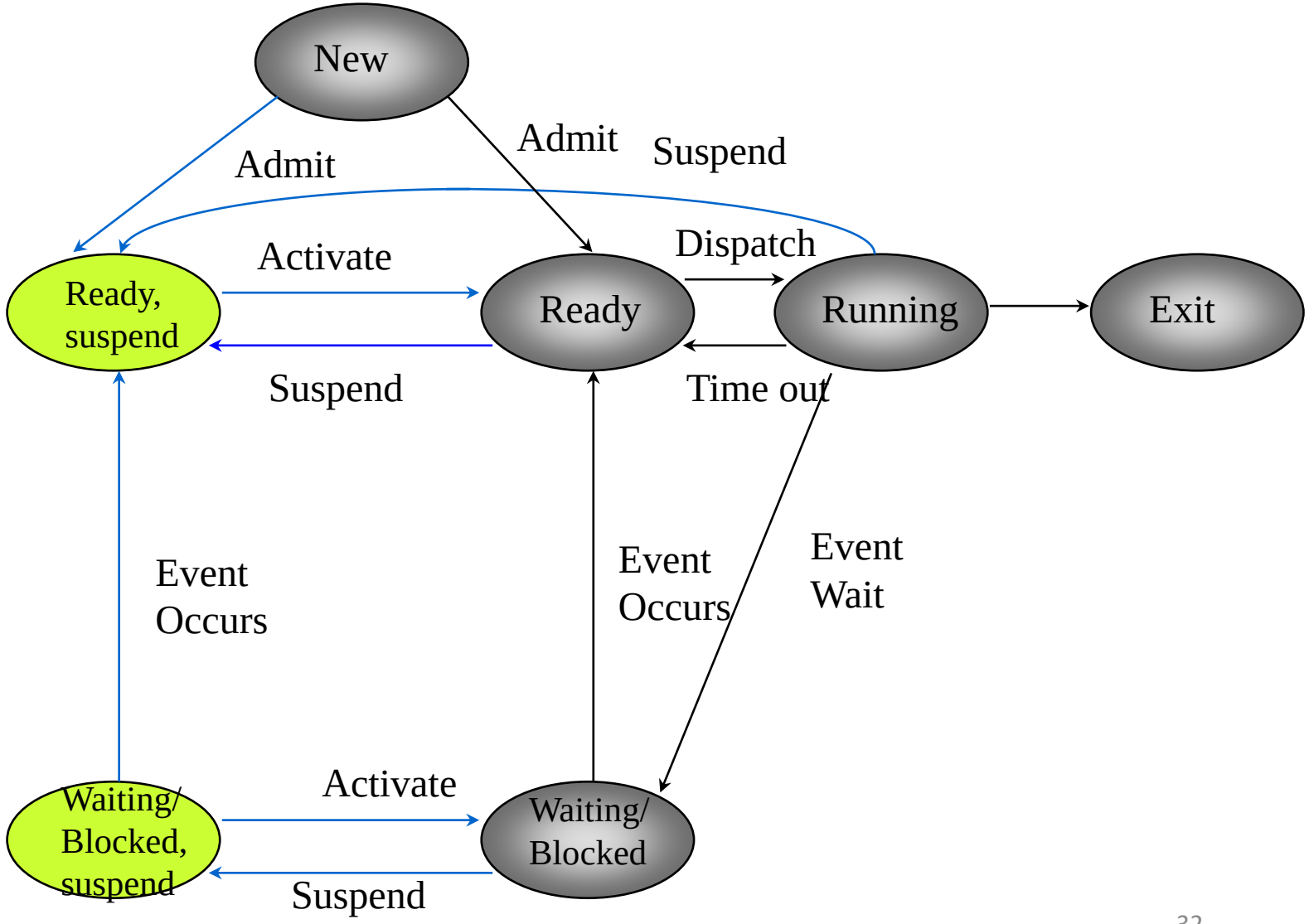
Process Durumları

- Kullanıcı matematiksel bir işlem için rakam girişleri yapıp eşittir tuşuna bastığında;
- **Bekleme-Bloke durumundan çıkar --> Hazır Kuyruğuna geçer --> Koşuyor durumuna geçer**.
- **Koşuyor durumunda** CPU'da işlemi gerçekleştirir ve sonucu ekrana yazar, process daha sonra tekrar **Bekleme-Bloke durumuna** geçer.
- Kullanıcıdan tekrar bir işlem bekler. **Bekleme-Bloke durumunda** olduğu sürece işlem yapmaz.

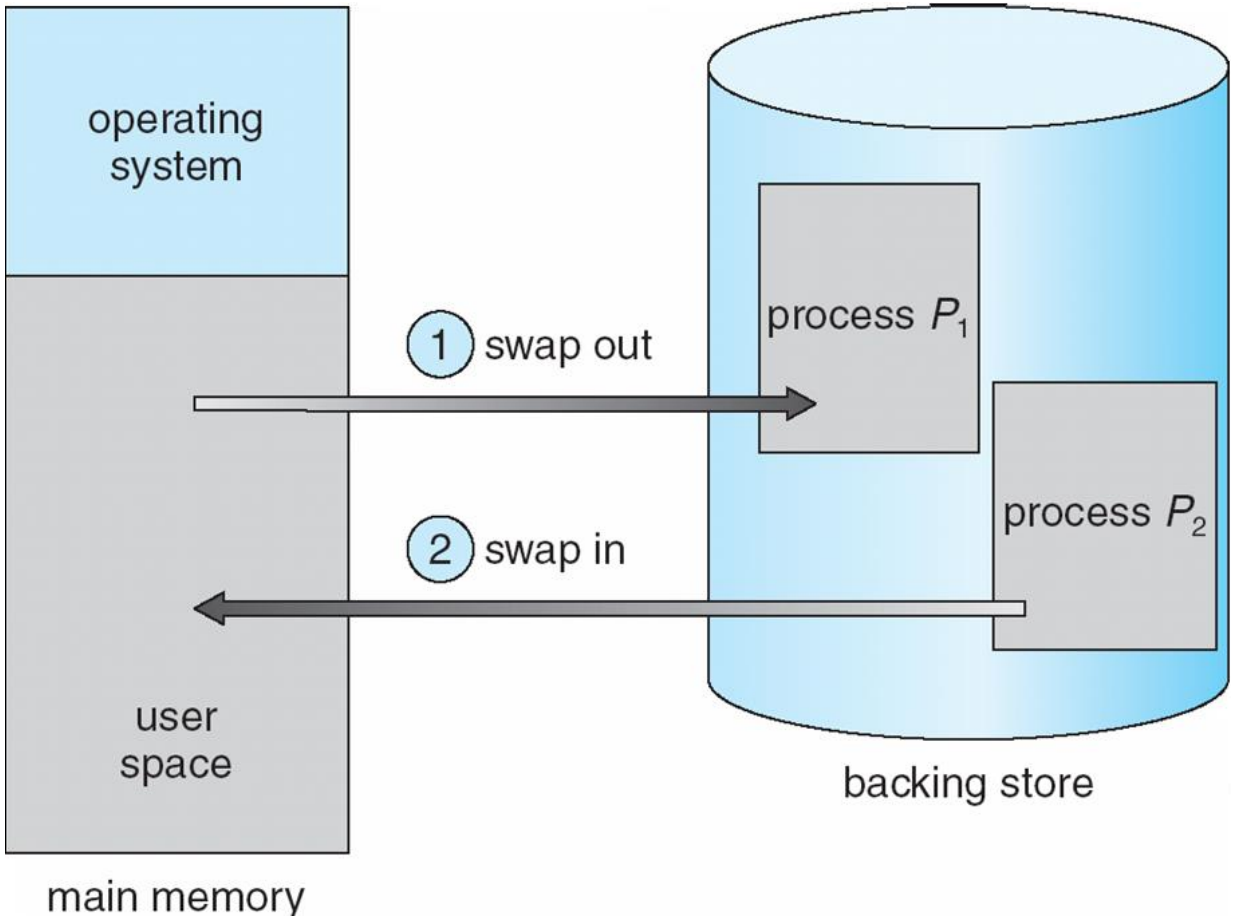
- Process'in sonlanması genellikle (fakat her zaman değil) **Koşuyor** durumdayken gerçekleşir.

Yedi Durumlu Process Modeli

- Bir process'i **askıya (suspend)** almak; bu process'i belleğin dışına çıkarma ve ikincil belleğe kaydetme durumunu içerir.
- Genellikle Swap (disk takası), işlemlerinde gerçekleşen bu olay iki yeni process durumunu gerektirir.



Disk Takası (Swapping)

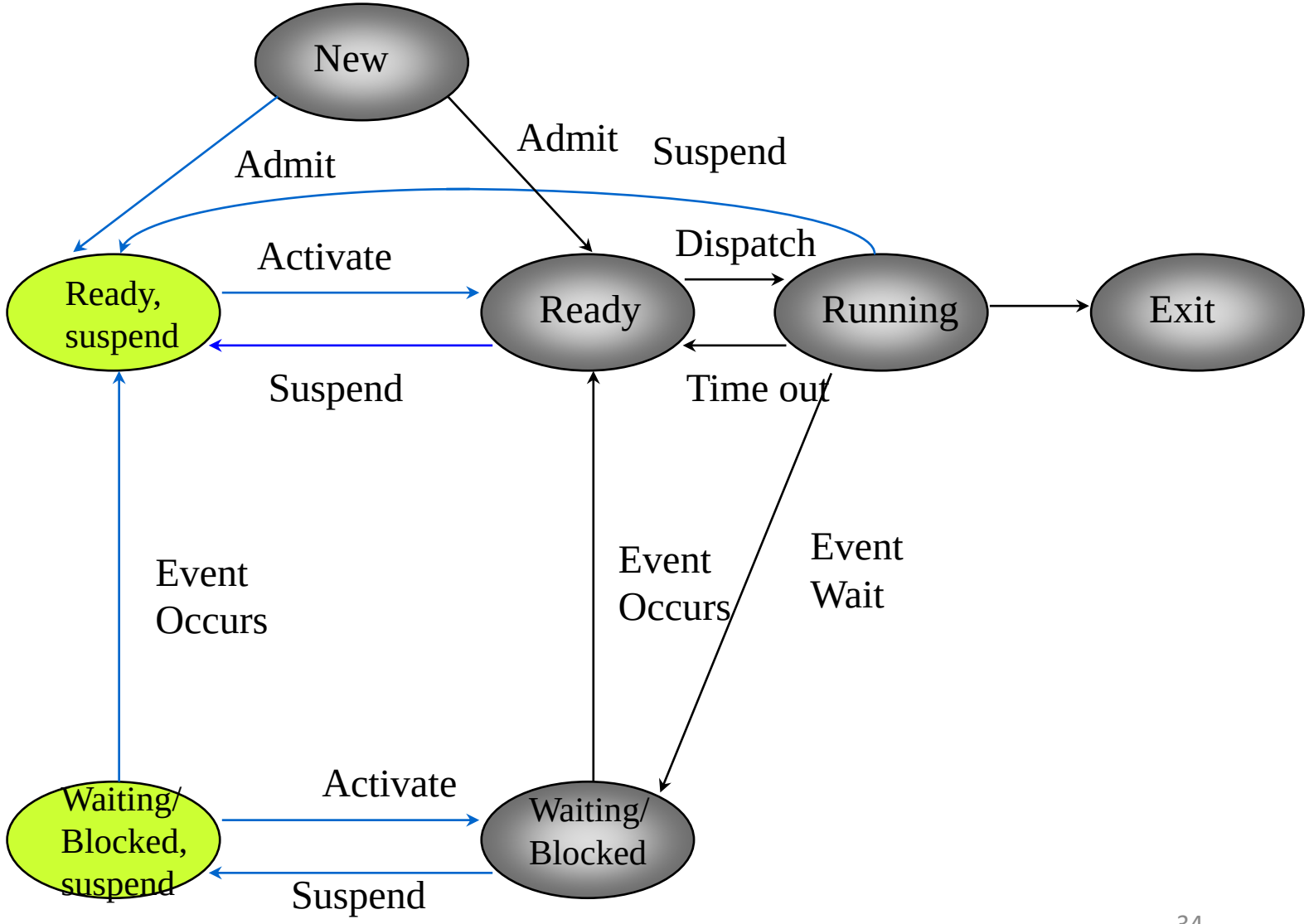


Yedi Durumlu Process Modeli

waiting/blocked-suspend : Process ikincil bellekte ve bir olay (event) için bekliyor.

ready-suspend : Process ikincil bellektedir, ancak ana belleğe yüklenir yüklenmez çalıştırılabilir.

Hazır (Ready) veya Bekle (Waiting) durumunda olan process'ler, sistemin kaynak talebine bağlı olarak Askıya (Suspend) alınabilir.



Process'in Askıya Alınma Nedenleri

- Process'ler bir çok nedenden dolayı askıya (suspend) alınabilir. Bunlardan bazıları;

- **Swap işlemi :** Yeni bir process koşturmak için bellekte boş alan yoktur bu yüzden işletim sistemi ek bellek alanına gerek duyuyor olabilir. Bir process'in tamamı veya bir bölümü ana bellekten diske taşınır.

- **Hatalı durum oluşması :** İşletim sistemi, bir soruna neden olduğundan şüphelenilen bir process'i askıya alabilir.
- **Etkileşimli kullanıcı isteği :** Bir kullanıcı, bir soruna neden olduğundan şüphelenilen bir process'i askıya almak isteyebilir.
  - –Örneğin hata ayıklama (debug) için
- **Parent Process'in Beklenmesi:** Eğer bir child process işleminde Parent Process'in bir işlemi yapması bekleniyorsa child process askıya alınır.

Yeni Durum Geçişleri

- **Waiting/Blocked –> Waiting/Blocked Suspend :**

Process'ler bekleme/bloke durumundayken, işletim sistemi bellekte yer açmak için process'leri ikincil belleğe alır ve bir olay (event) için bekler.

- **Waiting/Blocked Suspend –> Ready Suspend :**

Process hala ikincil bellektedir, ancak beklenen olay (event) gerçekleşmiş ve process ana belleğe yüklenir yüklenmez çalıştırılabilir.

- **Ready Suspend –> Ready :**

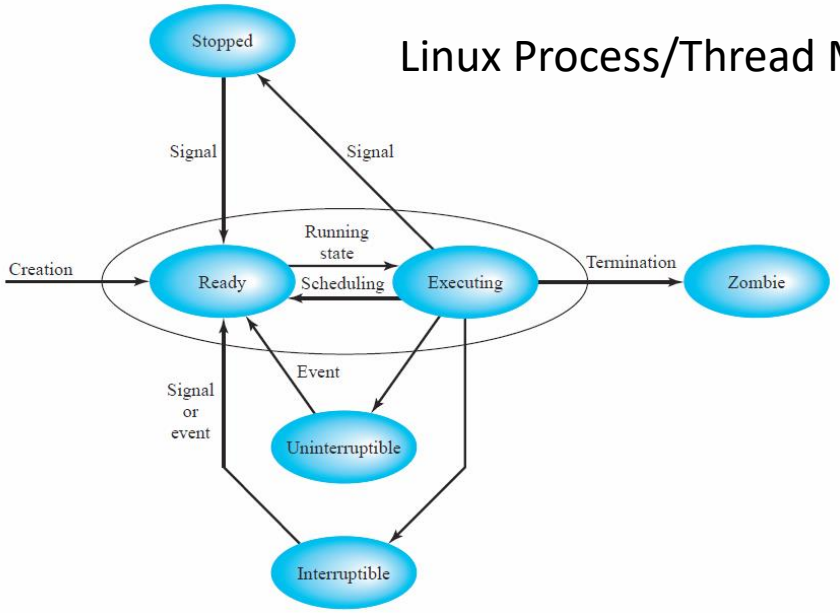
Process ikincil bellekten ana belleğe hazır kuyruğuna alınır.

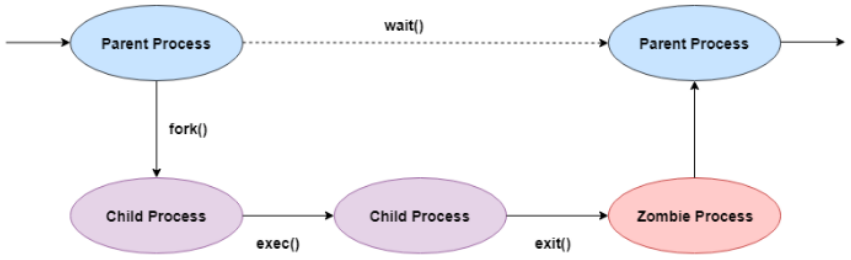
- **Ready –> Ready Suspend (nadiren gerçekleştirilir!!!!) :**

Bekleme durumunda process'ler mevcut olmadığında yeterli performansı sağlamak için bellek boşaltılır; bunun için bellekteki Hazır (ready) durumdaki process'ler ikincil belleğe alınır.

Zombi (Zombie) Process

- POSIX standartlı işletim sistemlerinde, sonlanmış ancak process tablosundan bilgileri silinmemiş process'e **zombi process** denir.
- Zombi process'ler ps komutu ile process listesi alındığında 'z' harfi ile gösterilir.
- Zombi process'ler kullandıkları tüm kaynakları iade etmişlerdir. Genellikle alt processler için meydana gelir.
- UNIX/Linux sistemlerinde bir alt process sonlandığında onu oluşturan parent process *wait* ya da *waitpid* sistem çağrısını kullanmazsa zombi process, process tablosunda *wait* işlemi yapılana kadar kalır.
- Zombi process'ler, kill komutu kullanılarak SIGCHLD sinyalini parent process'e göndererek sistemden kaldırılabilir. Bu sinyal, wait sistem çağrısını kullanarak zombi process'ini temizlemesi için parent process'i bilgilendirir.
- Zombi process'i hala parent process tarafından process tablosundan çıkarılmamışsa, parent process sonlandırılır.
- Parent process artık çalışmıyorsa, zombi process'inin varlığı bir işletim sistemi hatasını gösterir. Birkaç zombi process varsa bu ciddi bir sorun olmayabilir, ancak daha ağır yükler altında, sistem için process tablosu girişlerinin tükenmesi gibi sorunlar oluşturabilir.

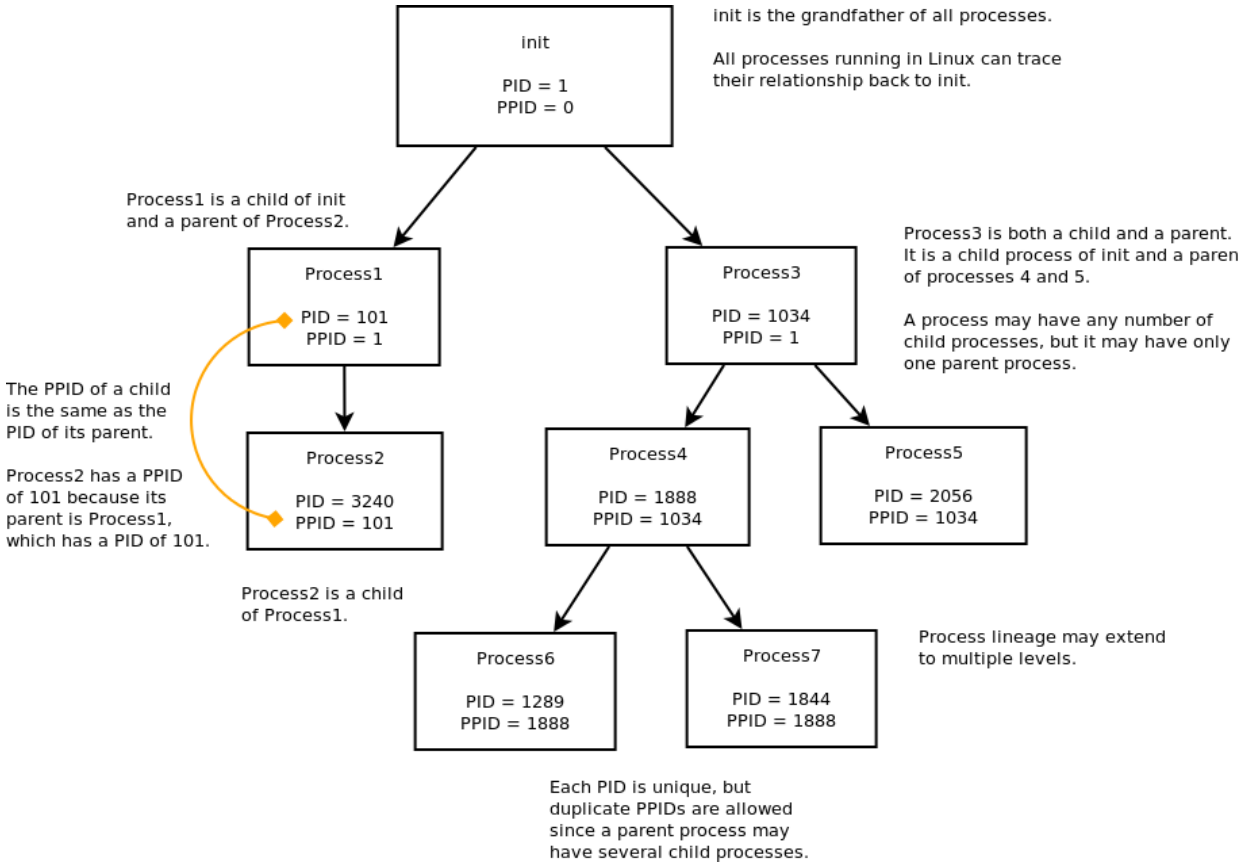




Zombie Process in Linux

Parent-Child Process İlişkisi

- Genellikle işletim sistemi yüklendiğinde bir başlangıç process'i otomatik olarak oluşturulur ve bu process tüm process hiyerarşisinin kökü olur. Bir process oluşturulduğunda bu köke eklenir.
- Çalışmakta olan bir program başka bir programı çalıştırabilir. Yani bir process başka bir process'i oluşturabilir.
- Bir process'in başka bir process'i oluşturması durumunda;
- Oluşturan process, üst process (**parent process**),
- Yeni oluşturulan process ise alt process (**child process**) olarak adlandırılır.

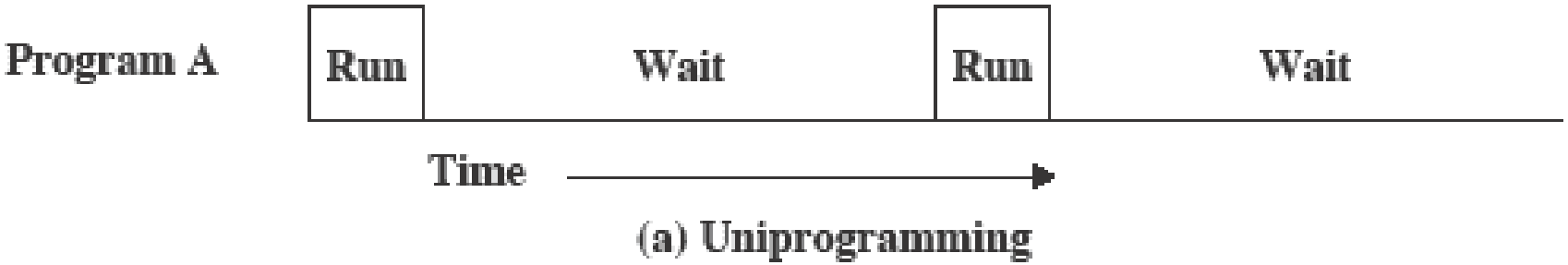


Parent-Child Process İlişkisi

- Alt process (child process) oluşturulur iken, üst process'den (parent process) bazı özellikleri almaktadır.
- Pek çok işletim sisteminde process'in erişim hakları, çalışma dizini, dosya betimleyicileri, çevre değişkenleri gibi özellikler yeni process oluşturma sırasında üst process'den alt process'e aktarılır.
- Alt process aktarılan özellikleri daha sonra değiştirmeden kullanabilir ya da değiştirebilir.
- Windows sistemlerinde alt-üst process ilişkisi UNIX/Linux sistemlerinden daha zayıftır, yani daha az özellik alt process'e aktarılır.

Tekli ve çoklu programlama (Uniprogramming and multiprogramming)

- Tek bir program için bir process'in yürütüldüğü durum (uniprogramming)

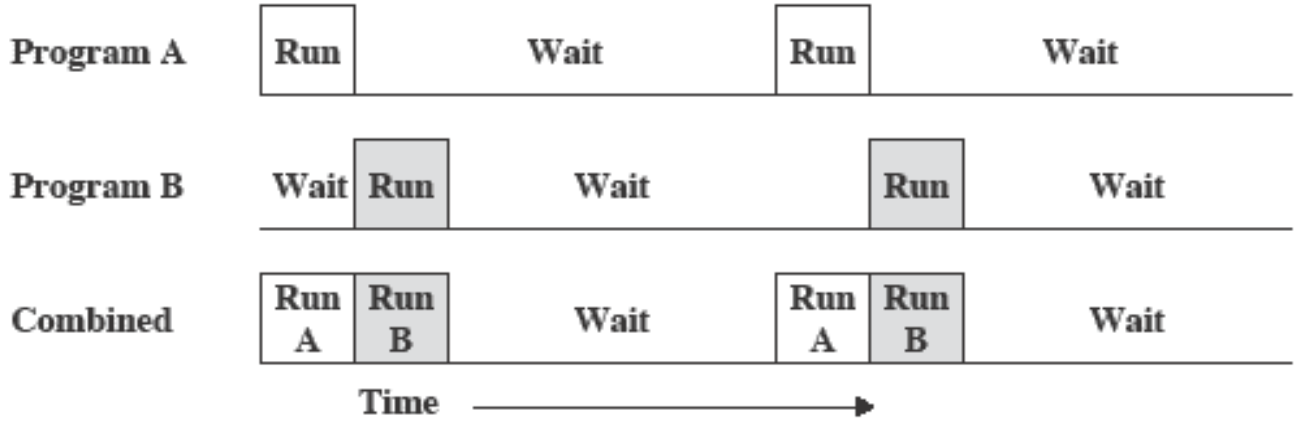


(a) Uniprogramming

- İşlemci, bir I/O komutuna ulaşana kadar belirli bir süreyi yürütmeye harcar; daha sonra devam etmeden önce I/O komutunun sonuçlanmasını beklemelidir.

Tekli ve çoklu programlama (Uniprogramming and multiprogramming)

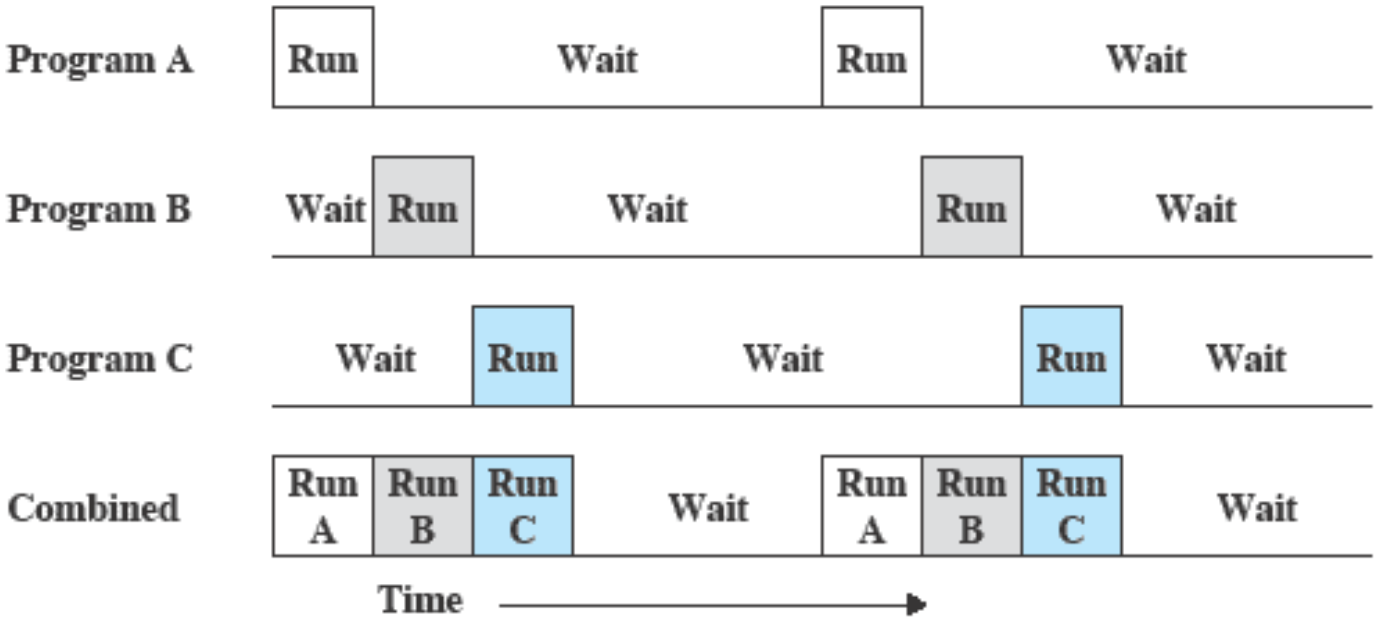
- Birden fazla program için birden fazla process'in yürütüldüğü durum (multiprogramming/multitasking).



(b) Multiprogramming with two programs

- Bir process'in I/O beklemesi gerektiğinde işlemci, büyük olasılıkla I/O beklemeyen diğer process'e geçebilir.

Tekli ve çoklu programlama (Uniprogramming and multiprogramming)



(c) Multiprogramming with three programs

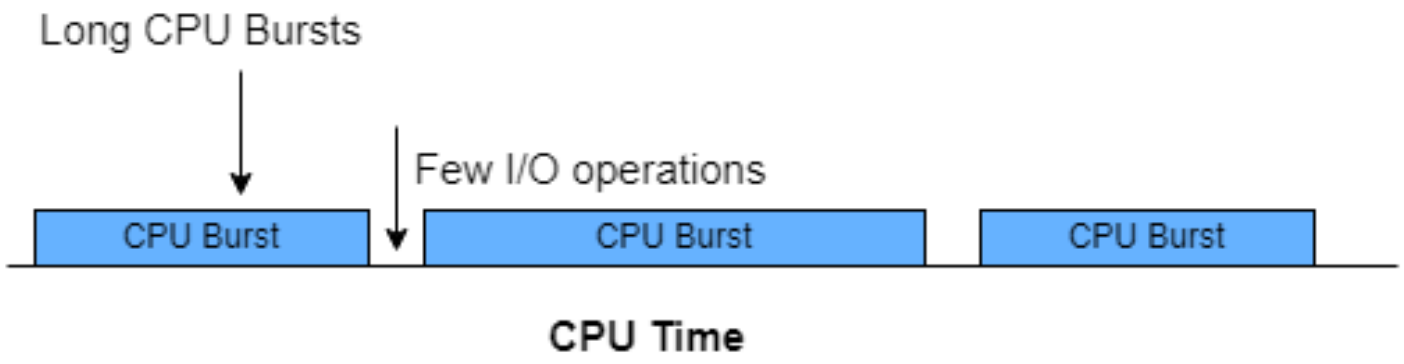
I/O ve CPU Bağımlı Process'ler

- Process'ler yürütülme açısından şu şekilde sınıflandırılır.
- **CPU bağımlı (CPU bound)** bir process daha fazla CPU zamanı gerektirir veya çalışma durumunda daha fazla zaman harcar.
- **I/O bağımlı (I/O-bound)** bir process daha fazla I/O süresi ve daha az CPU süresi gerektirir. I/O bağımlı bir process bekleme durumunda daha fazla zaman harcar.

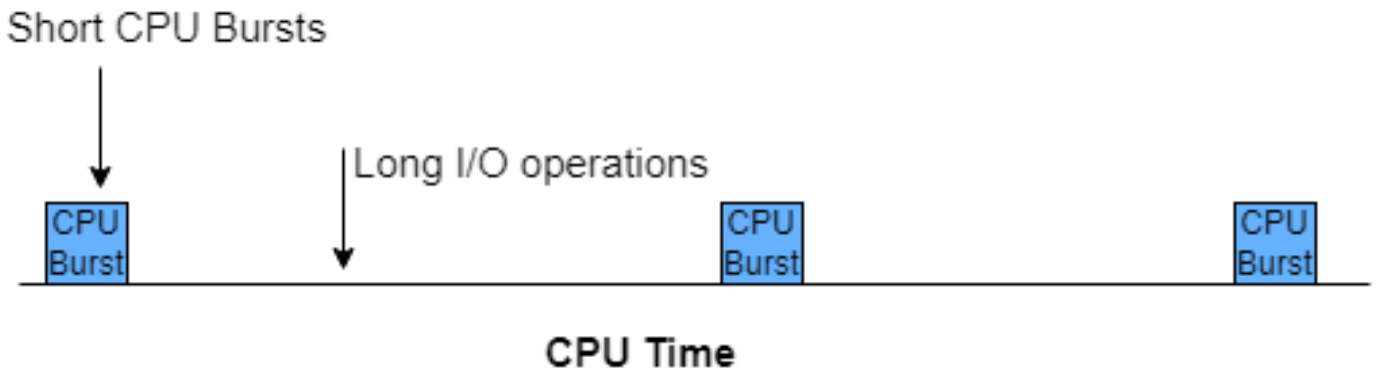
- Birçok gerçek dünya uygulaması CPU bağımlı ve I/O bağımlı yürütmenin bir karışımına sahiptir.

I/O ve CPU Bağımlı Process'ler

- **CPU bağımlı (CPU bound)**



- **I/O bağımlı (I/O-bound)**



I/O ve CPU Bağımlı Process'ler

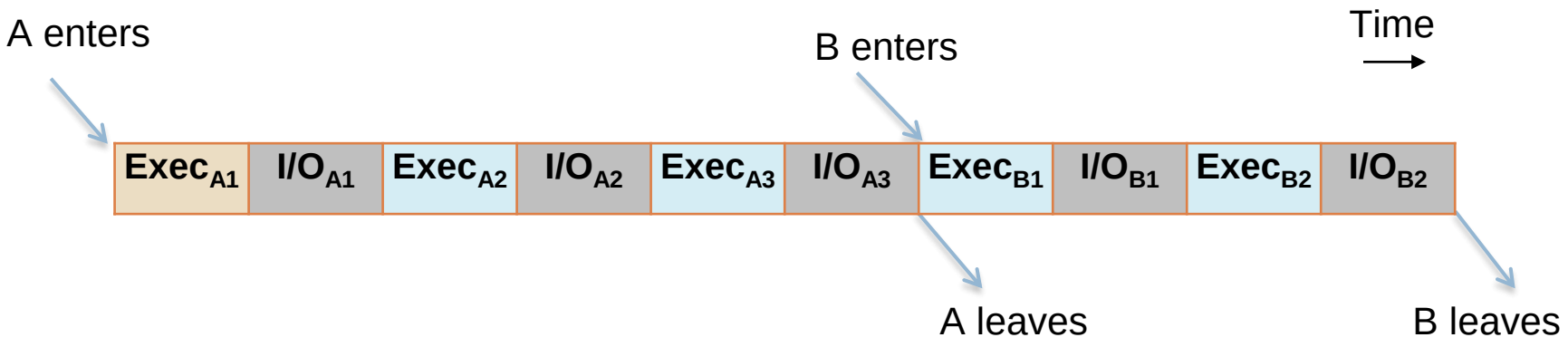
- A ve B olmak üzere iki process olduğunu varsayalım.
- Her ikisi de 1 saniye boyunca yürütülür ve 1 saniye boyunca bazı I/O işlerini yapar.
- Bu model A process'i için 3 kez, B process'i için 2 kez tekrarlanmış olsun.

$Exec_{A1}$	I/O_{A1}	$Exec_{A2}$	I/O_{A2}	$Exec_{A3}$	I/O_{A3}

$Exec_{B1}$	I/O_{B1}	$Exec_{B2}$	I/O_{B2}

I/O ve CPU Bağımlı Process'ler

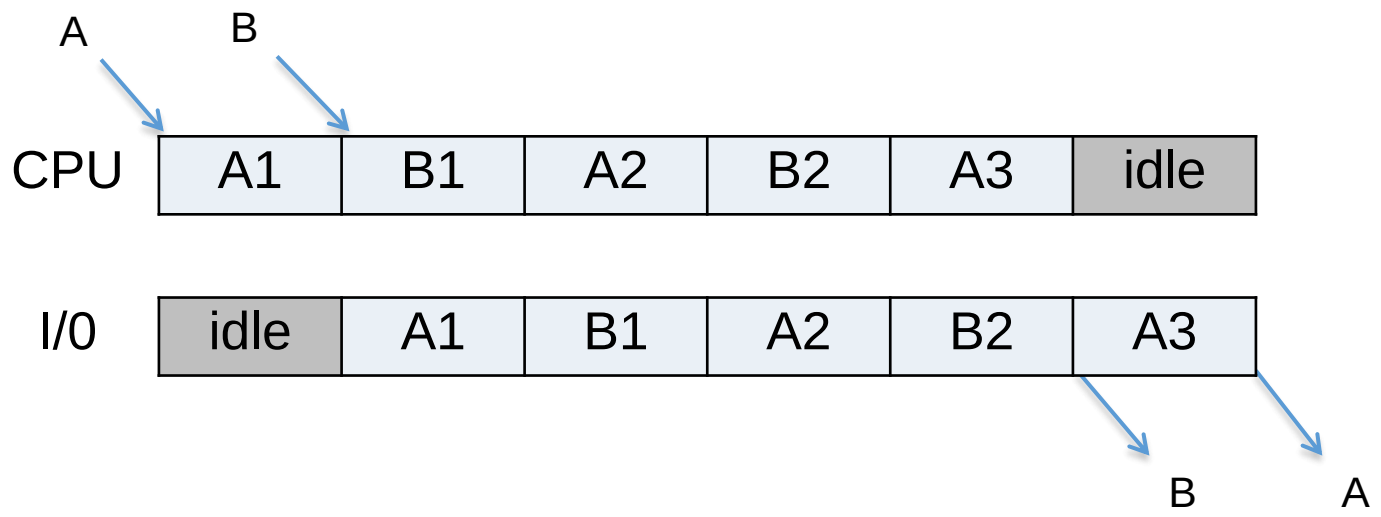
- Eğer işletim sistemi çoklu programlamayı (multiprogramming) desteklemiyorsa process'ler aşağıdaki gibi sıralı olarak yürütülür.



- İşlemci bu iki process'i toplam 10 saniye gibi bir sürede gerçekleştirir. Ancak process'lerin I/O anlarında boştadır. Yani işlemci 5 saniye boşta kalır ve 5 saniye kullanılır. İşlemci kullanımı %50.

I/O ve CPU Bağımlı Process'ler

- Şimdi çoklu programlama durumunu ele alalım:



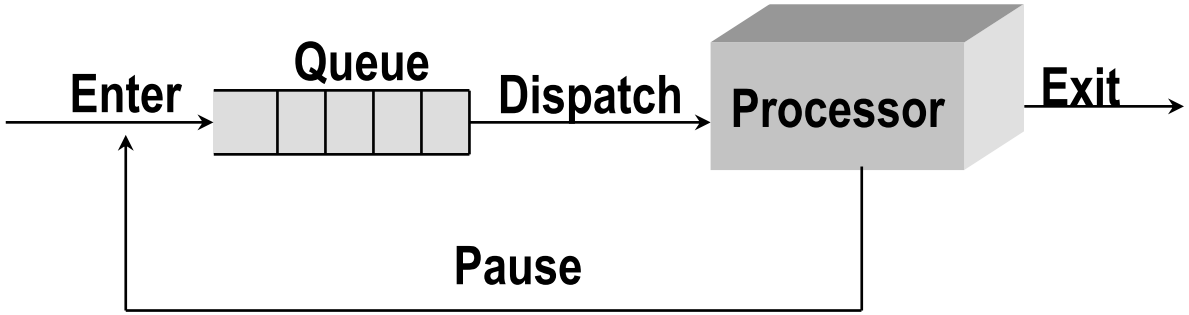
- Bu durumda, A process'i bazı I/O çalışmalarına geçtiğinde (yani işlemciyi kullanmadığında), işlemci boşta kalmak yerine zamanını B process'ini yürütmek için kullanır. İşlemci kullanımı %83.

Process Kuyruğu (Process queue)

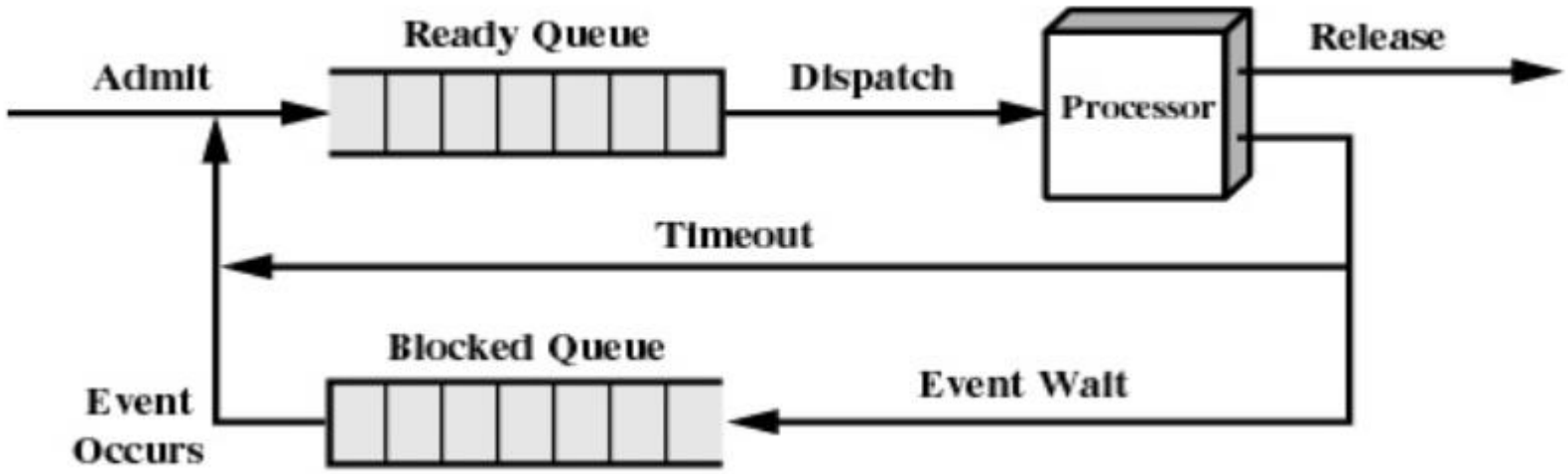
Basit bir sistemde tek bir kuyruk process'leri biriktirmek için yeterlidir.

Bir process ya aktiftir (running) yada pasiftir(waiting)

O anda koşturulmayan process sırasını bir kuyrukta bekler.

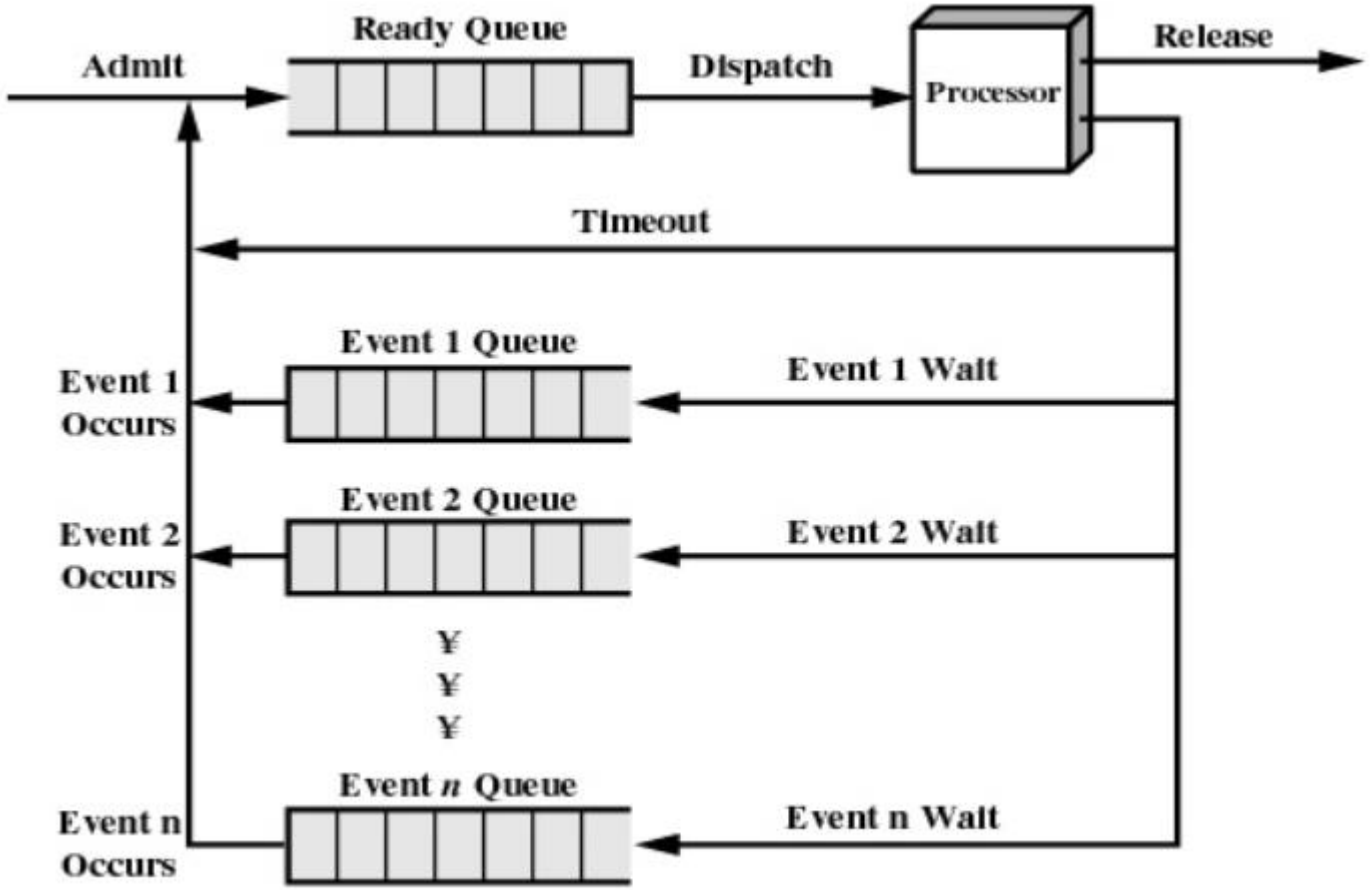


Tek Düzeyli Process Kuyruğu



Ancak tek bir bekleme/bloke kuyruğu kullanıldığında arka sıradaki bir process'in beklediği G/Ç işlemi geldiği halde öndeki process'leri beklemek zorundadır.
Bu problemin çözümü her olay (event) için bir bekleme/bloke kuyruğu oluşturulacak şekilde kuyruğun yatayda büyütülmesidir.

Çok Düzeyli Process Kuyruğu



Kuyrukta yüzlerce process olduğu düşünüldüğünde her bir olay için bir kuyruk tahsisi yapmak daha verimli bir kullanım olacaktır. İşletim sistemleri genellikle her olay için ayrı bir bekleme/bloke kuyruğu oluşturmaktadır.

Çok Düzeyli Process Kuyruğu

- Çok düzeyli process kuyruğu yaklaşımında, Process'ler 'bekleme/bloke (waiting/blocked)' durumundayken, rastgele bir kuyruğa girmez.

- Sadece kendi ile ilgili olan kuyruğa girer.

- Çünkü bazı process'lerin çalışması daha uzun sürebilir bazı process'lerin çalışması daha kısa sürebilir.

Çok Düzeyli Process Kuyruğu

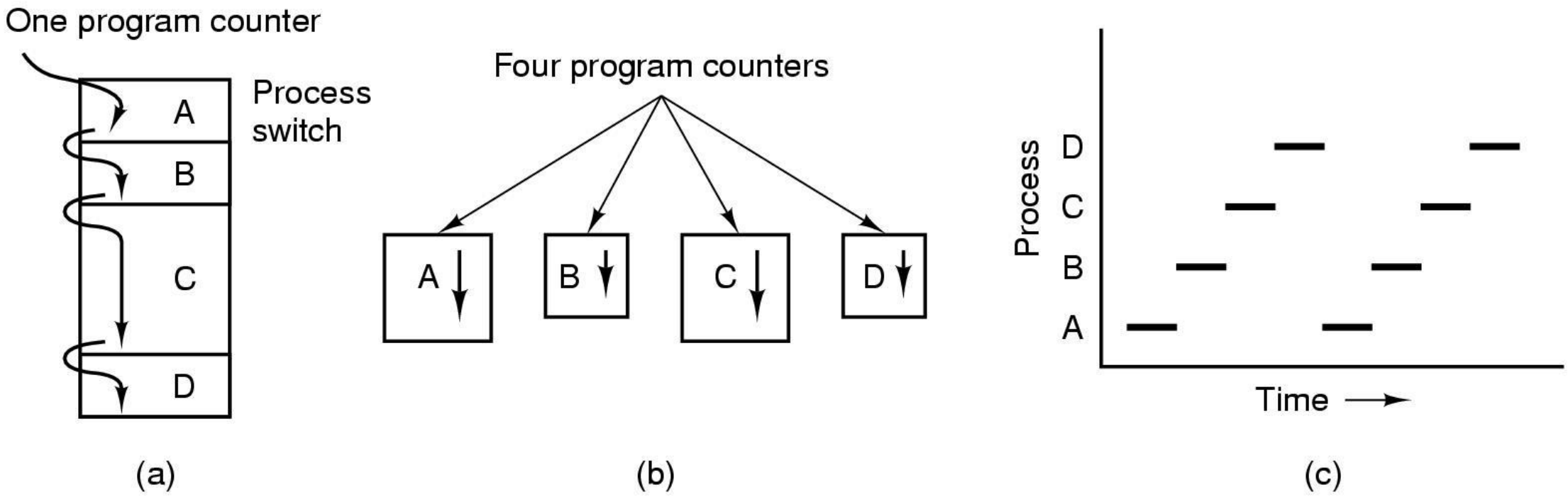
- Tüm process'ler aynı kuyruğa girerse, açılması daha kısa süren program, açılması daha geç olan programında arkasında sıraya girer ise, 1 saniyede açılacak olan program 10 saniyede açılabilir.

- İşletim sistemi bu problemi önlemek için, benzer işlemleri yapan process'leri bilgisayar ana belleğinde farklı farklı yerlerde kuyruğa alır.

Process Anahtarlama

- CPU'yu mevcutta kullanan bir process genelde şu durumlarda anahtarlanır/değiştirilir:
- Çoklu görev (Multitasking) :
  - Bir process kendisine ayrılan zaman dilimi kadar çalışmıştır saat kesmesi (clock interrupt) ile CPU başka bir process'e tahsis edilir.
  - Yüksek öncelikli bir process hazır duruma geldiğinde, çalışan process durdurulur ve daha yüksek öncelikli process yürütmesi için CPU'ya verilir.
- Kesme işlemi (Interrupt Handling) :
  - Process bir veri giriş/çıkış ihtiyacında olabilir. (I/O interrupt)
  - İstenen bellek adresi sanal bellekte olduğu için ana belleğe getirilmesi gerekir. (Memory Fault)
  - Komuttan kaynaklanan bir hata, process'in sonlandırılmış durumuna taşınmasına neden olabilir.
- Kullanıcı ve Çekirdek Modu Değiştirme (Kernel/User Switch) :
  - Kernel ve user mod geçişleri. Sistem çağrısı (System call)

Process Anahtarlama



Bağlam anahtarlama (context switching)

- İşlemcinin, process'leri görünürde paralel yürütmesini sağlamak üzere bir process koşturmasını bırakıp diğer bir process yürütmesine geçmesi belirli önlemler alınmadan yapılamaz.
-
- Process yürütmesinin yarım bırakıldığı andaki process durum bilgilerinin saklanması gerekir.

- Bir process anahtarlanmasından sonra process'in tekrar tutarlı bir biçimde kaldığı yerden CPU'da yürütülmeye devam edilmesi için process anahtarlamanın olduğu anda process'in durum bilgileri saklanır.

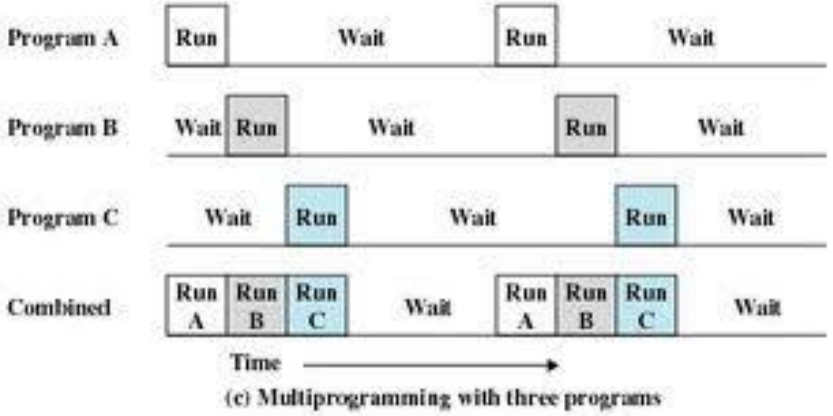
Bağlam anahtarlama (context switching)

- CPU'nun bir process yürütmesini, daha sonra devam etmek üzere bırakıp diğer bir process yürütmesine geçmesi **bağlam anahtarlama (context switching)** olarak adlandırılır.

- Bu nedenle her process için işletim sistemi tarafından bir veri yapısı tutulur.
- Bu veri yapısı CPU'da process koşturmasının hangi komuttan başlayarak sürdürüleceği bilgisini, işlemci yazmaç (register) içeriklerini, varsa kullanılan dosyalarla ilgili bilgileri içermek zorundadır.

Bağlam anahtarlama (context switching)

- CPU anlık olarak tek bir process'i yürütebilir.
- Aslında anlık olarak CPU'da tek process yürütüp birden fazla process yürütülüyor gibi gösteren işletim sistemi özelliği kısaca çok görevlilik (multitasking) olarak adlandırılır.
- Bu özellik CPU'nun, process'leri zaman paylaşımlı olarak yürütmesi sayesinde elde edilir.
- Process'ler arasında CPU'nun geçiş yapması esnasında bağlam anahtarlama (**context switching**) ile, yürütümü durdurulan process bağlamı (context) kaydedilir, yeni yürütülecek process'in de bağlamı CPU'ya yüklenir.



(c) Multiprogramming with three programs

Bağlam anahtarlama (context switching)

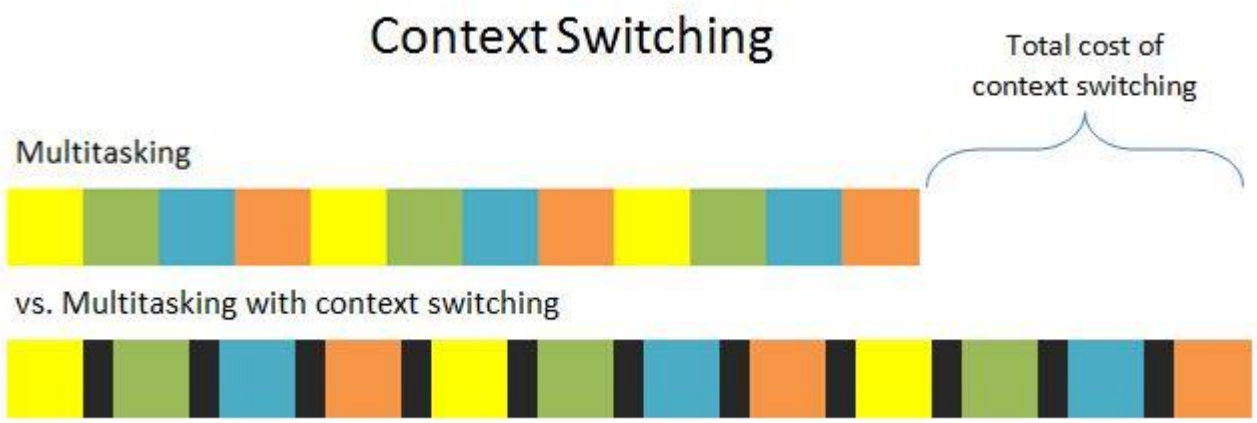
- CPU, bir process'den diğerine anahtarlandığında data ve durum kaydedicilerinin içeriklerinin kaydedilmesi gereklidir.
- Örneğin 1 birimlik bilginin yüklenmesi için 20 nanosaniyeye ihtiyaç duyulduğu ve 50 tanede kaydedici olduğu düşünülürse, bu kaydedicilerin içeriklerinin kaydedilmesi 1 mikrosaniye alacaktır.
- Ayrıca yeni yüklenecek process'in kaydedicileri de 1 mikrosaniye'de CPU'nun kaydedicilerine aktarılacağından bu süreç 2 mikrosaniye sürecektir.
- Bu süre zarfında başka process'ler yürütülebilecekken, bağlam anahtarlama (context switching) işlemine zaman ayrılması gerekmektedir.

Bağlam anahtarlama (context switching)

- Bir process anahtarlaması gerçekleştiğinde, kalınan noktadan itibaren process'in tekrar devam edebilmesi için o anki process'e ait tüm bağlamı (context) saklaması ve sonradan tekrar yükleyebilmesi gerekir.
- İşletim sistemi yürütmesi durdurulan process'in o anki haline ait bağlamı (context) saklamak için Process Control Block (PCB) yapısını kullanır. PCB yapısı, **Process image** olarak da adlandırılır.
- Process'in ilerde CPU üzerinde yürütmesini devam ettirebilmek için PCB'ye process bağlamı kaydedilir.
- Sıra tekrar aynı process'e gelince de PCB'den process'e ait bağlam tekrar CPU'ya yüklenir ve kalınan yerden yürütülmeye devam edilir.

Bağlam anahtarlama (context switching)

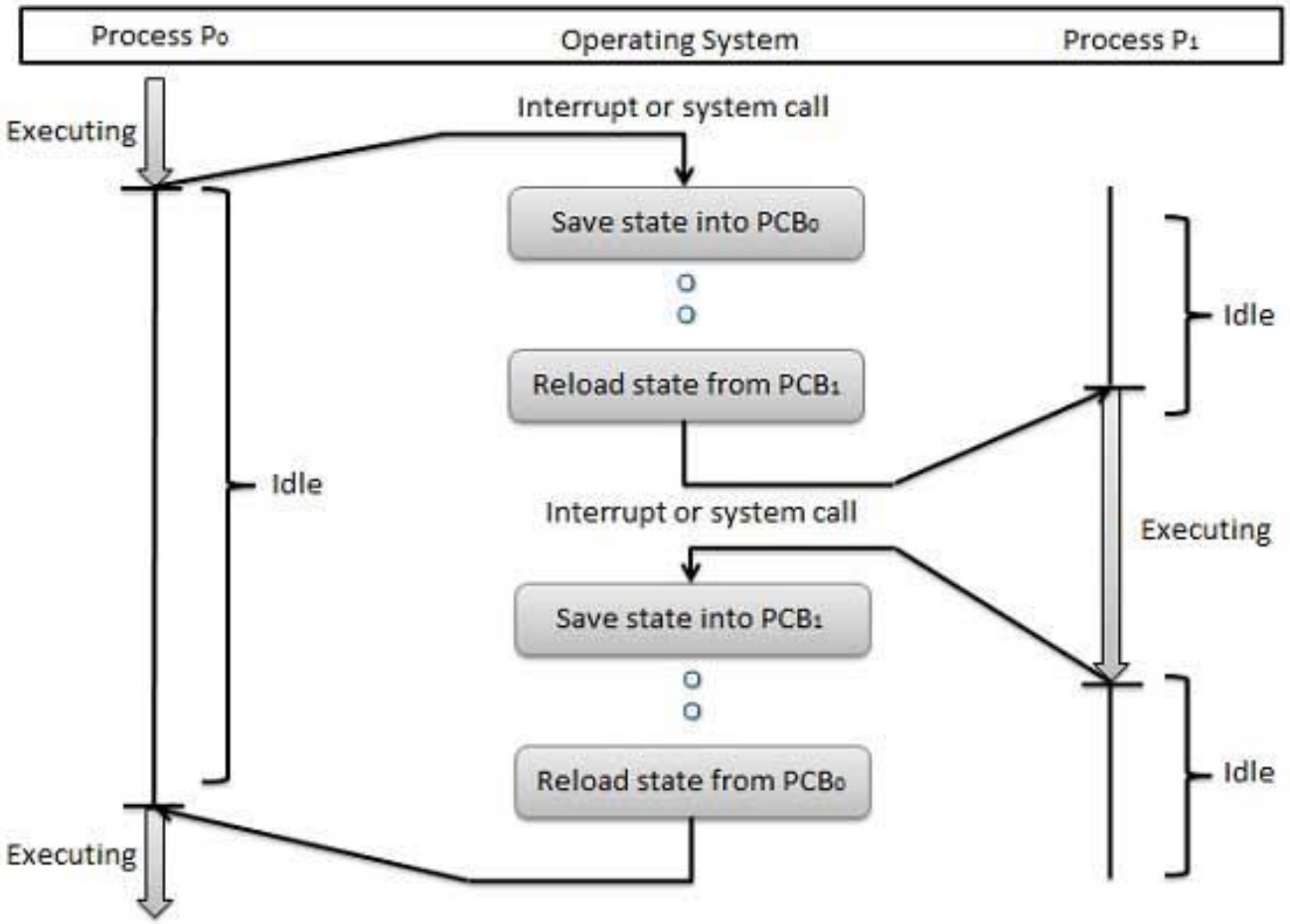
- Çoklu process koşturma tekniğinde, CPU'da, bir process'den diğer bir process'e geçerken, periyodik olarak **bağlam anahtarlama (context switching)** gerçekleşir.
- Aslında her **bağlam anahtarlama** işleminin bilgisayar açısından bir maliyeti vardır.
- Dolayısıyla **bağlam anahtarlama** aslında iyi bir özellik olmasına karşılık bir de maliyeti vardır ve oranı yükseldikçe dezavantaj haline gelir.



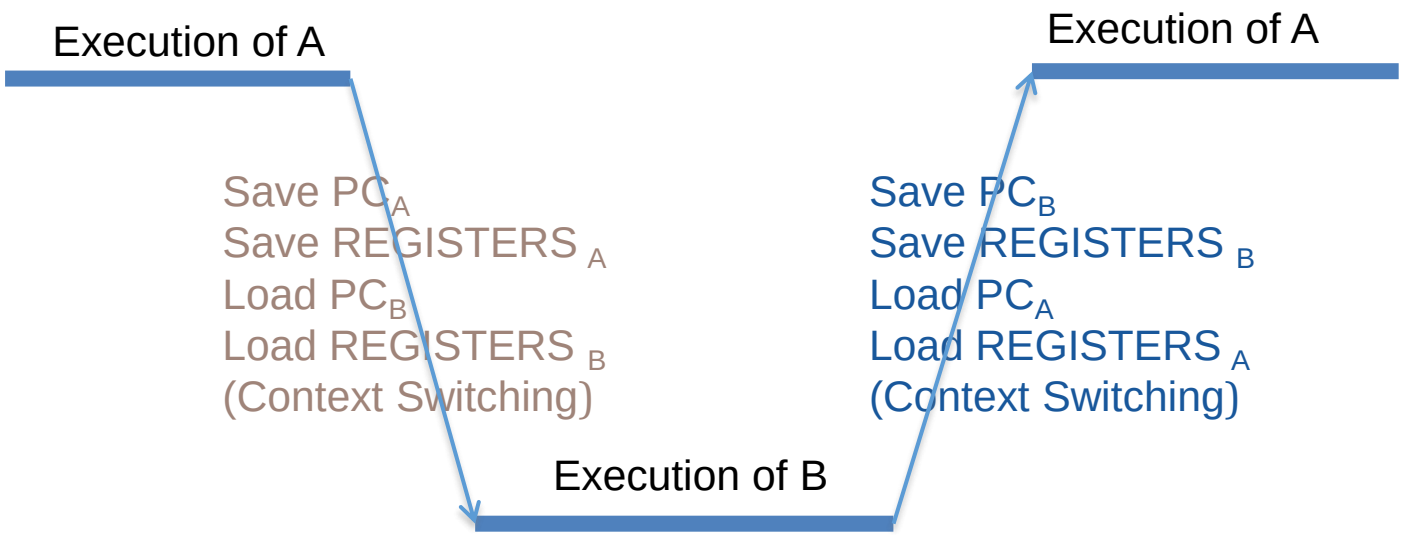
Bağlam anahtarlama (context switching)

- Bağlam anahtarlama sırasında process’e ait bağlam (context) sürekli olarak ana bellekte PCB’ye yazılır ya da okunurlar.
- Bu nedenle ana bellek üzerinde sürekli olarak okunup yazılma işlemi gerçekleştirilir, Bu iki durumda istenmeyen durumlardır.
- Gerçekleşen bu **bağlam anahtarlama** işleminden dolayı belirli bir zaman kaybı olur.
- İyi bir işletim sisteminde **bağlam anahtarlama** sayısının düşük olması istenir.

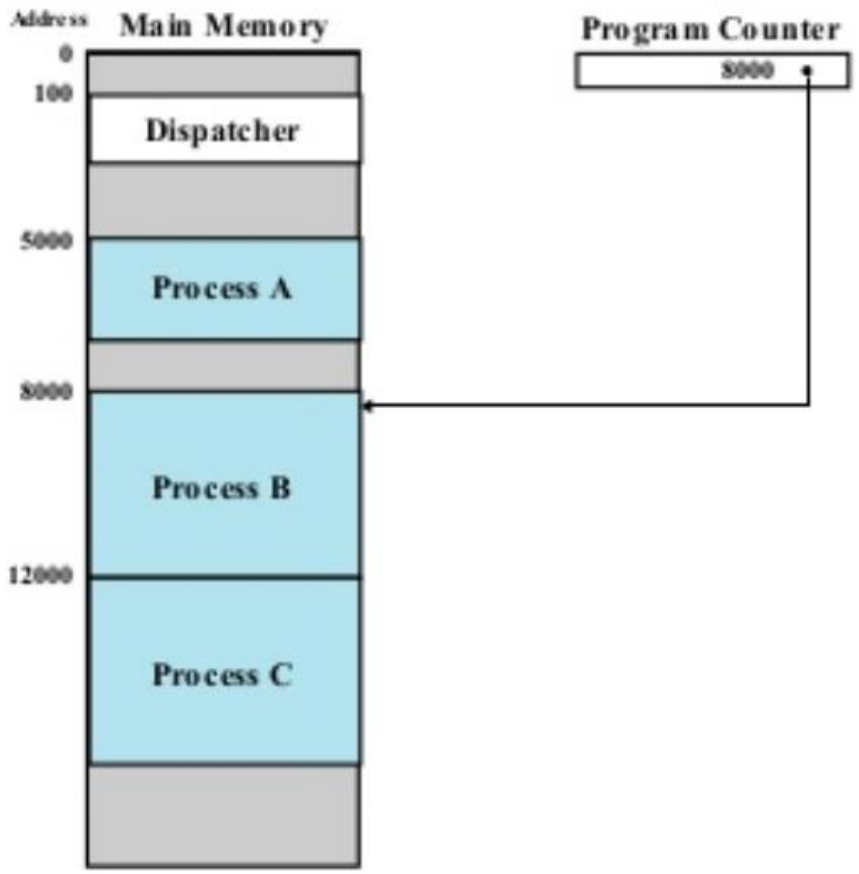
Bağlam anahtarlama (context switching)



Bağlam anahtarlama (context switching)



Process Koşturmasının İzlenmesi



Program Counter, bir işlemcinin çalıştırdığı programda işlem yapılacak olan sonraki komutun bellekte nerede olduğunu gösteren bir kayıttır.

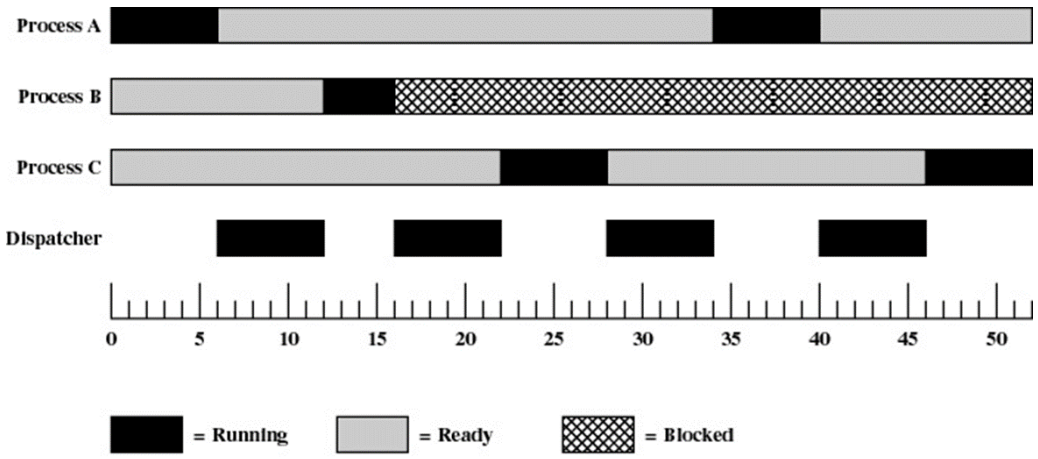
İşlemci, her bir komutu Program Counter kaydında tutulan bellek adresine giderek alır ve o adreste bulunan komutu işler.

Daha sonra Program Counter kaydı bir sonraki komutun bellek adresini gösterir ve işlemci bu adrese giderek işlem yapmaya devam eder.

Process Koşturmasının İzlenmesi

(a) Trace of Process A	(b) Trace of Process B	(c) Trace of Process C
5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

5000 = Starting address of program of Process A
8000 = Starting address of program of Process B
12000 = Starting address of program of Process C



1	5000		27	12004	
2	5001		28	12005	
3	5002		----------		Time out
4	5003		29	100	
5	5004		30	101	
6	5005		31	102	
----------		Time out	32	103	
7	100		33	104	
8	101		34	105	
9	102		35	5006	
10	103		36	5007	
11	104		37	5008	
12	105		38	5009	
13	8000		39	5010	
14	8001		40	5011	
15	8002		----------		Time out
16	8003		41	100	
----------		I/O request	42	101	
17	100		43	102	
18	101		44	103	
19	102		45	104	
20	103		46	105	
21	104		47	12006	
22	105		48	12007	
23	12000		49	12008	
24	12001		50	12009	
25	12002		51	12010	
26	12003		52	12011	
			----------		Time out

100 = Starting address of dispatcher program

shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed

İşletim Sistemi Kontrol Tabloları (OS Control Tables)

İşletim sistemi tarafından yönetilen her varlık için tablolar tutulur.

- Bellek Tabloları
- G/Ç Tabloları
- Dosya Tabloları
- Process Tabloları

Her process ve kaynak ile ilgili durum bilgilerinin tutulması gerekir.

İşletim sistemi process'lerle ilgili gerekli bilgileri bir process tablosu (process table) içinde muhafaza eder.

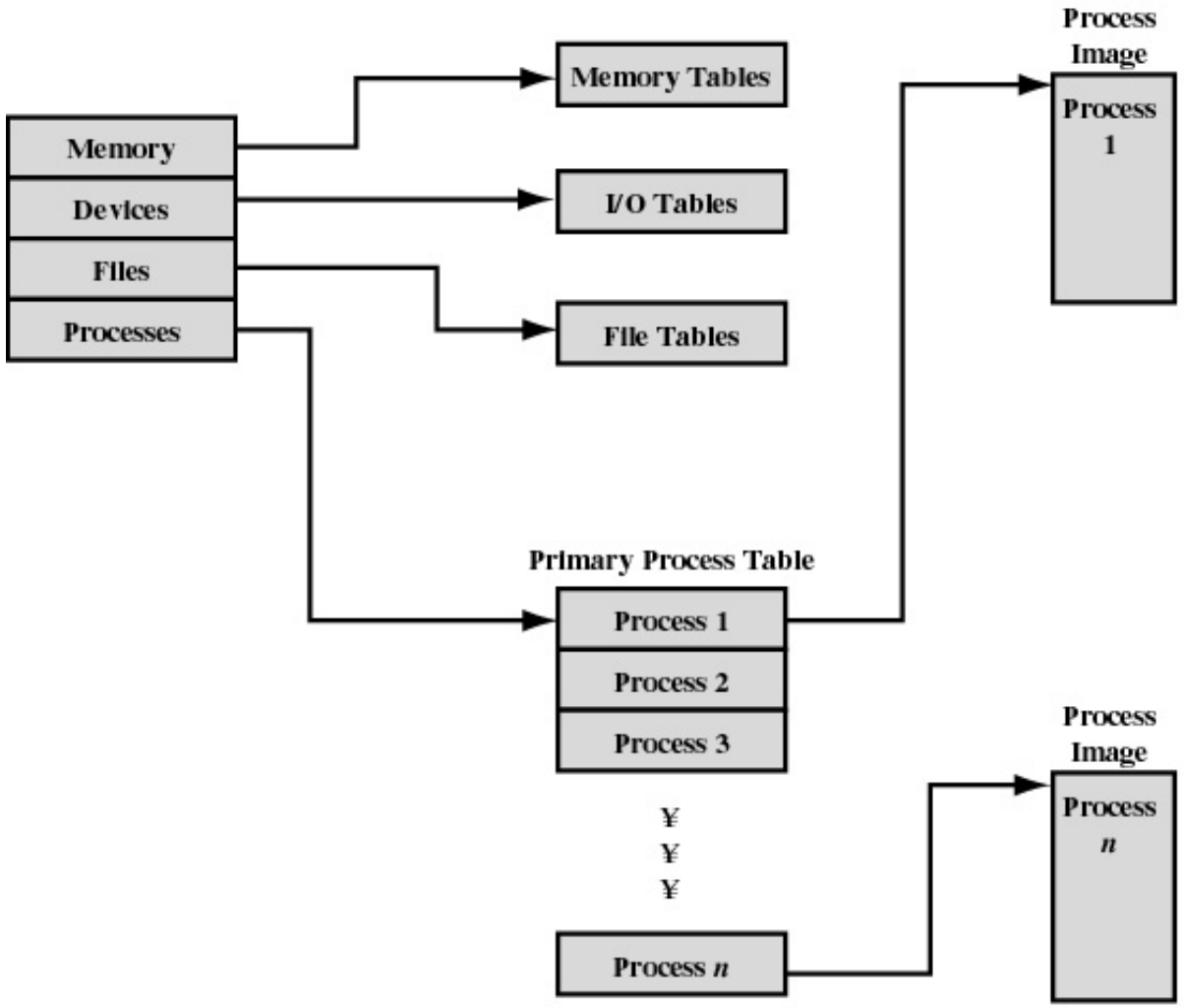
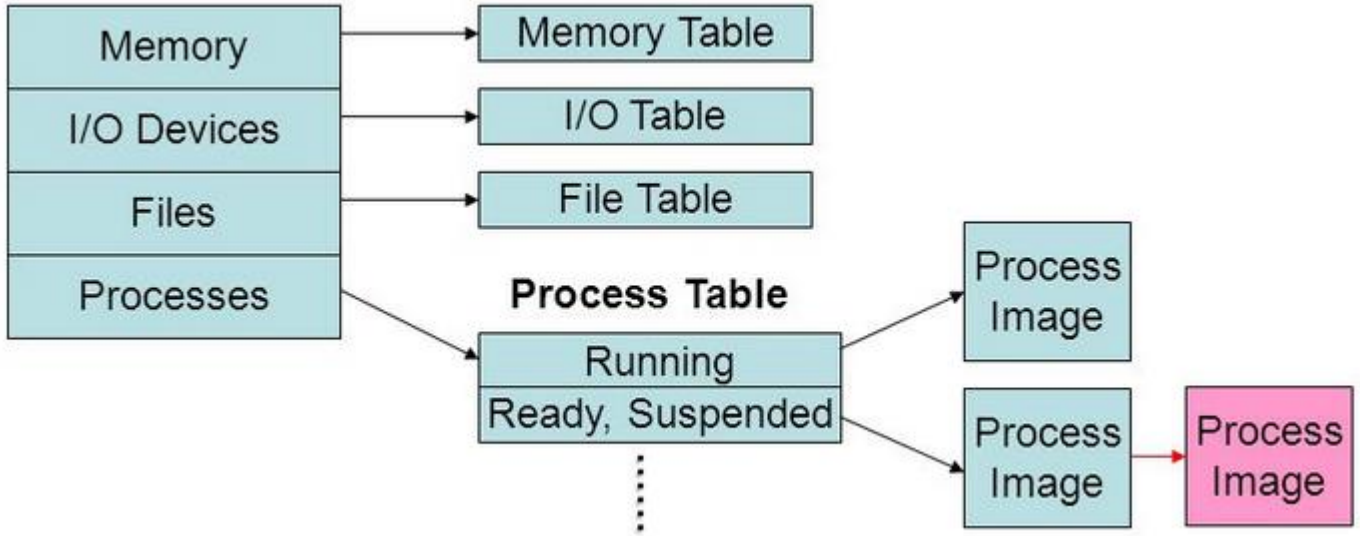


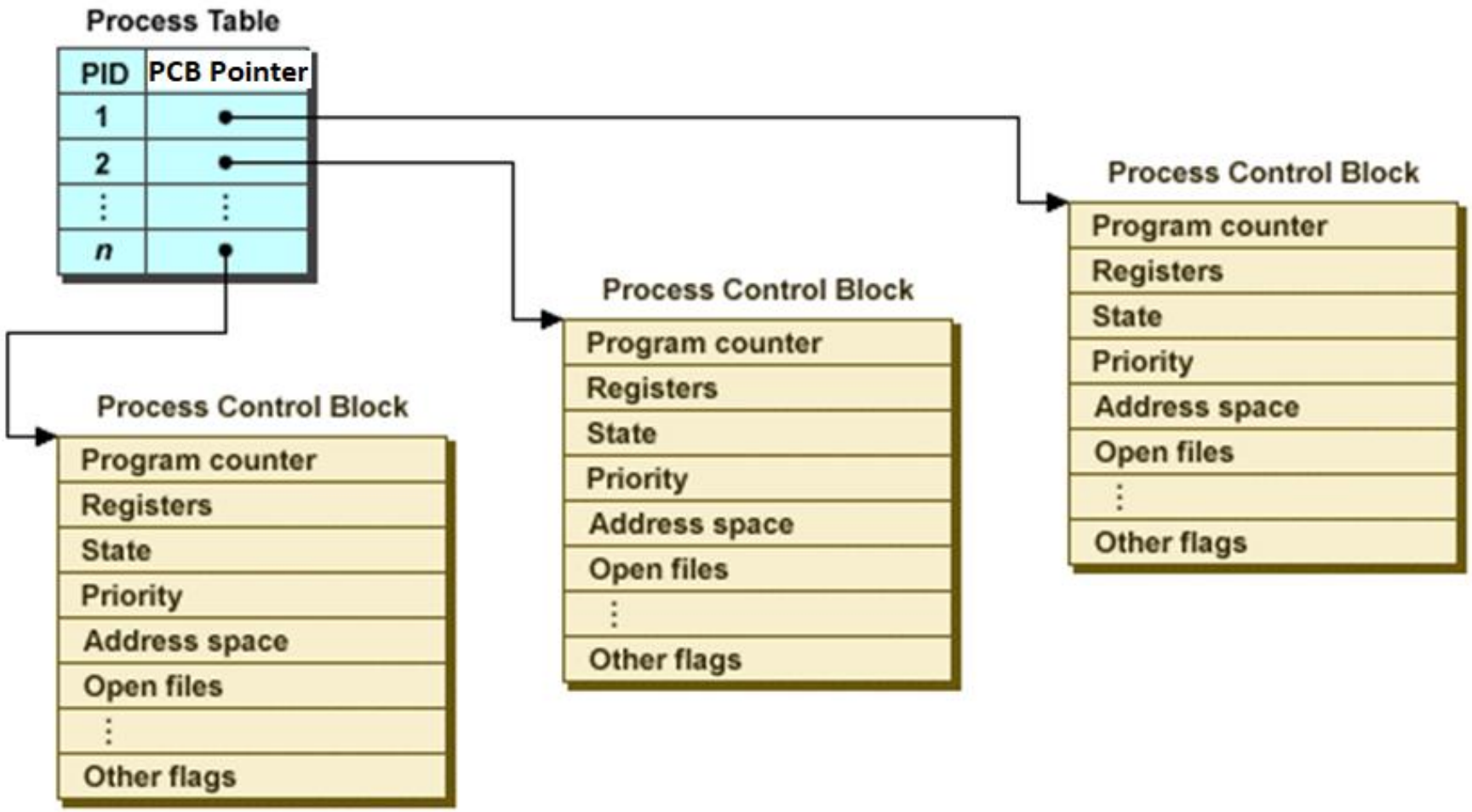
Figure 3.10 General Structure of Operating System Control Tables

Process Table ve Process Control Block/Process Image



- Process'lerin gerçekleştirilmesi sırasında işletim sistemi o process'e bir process kimliği (process ID) vererek process tablosuna ekler.
- Process tablosunda her process'in görüntüsüne (process image) ilişkin işaretçi bilgisi ile görüntünün olduğu belleğin başlangıç adresi saklanmaktadır.

Process Control Block (PCB) - Process Image



Her process, işletim sisteminde process control block (PCB)) tarafından temsil edilir.

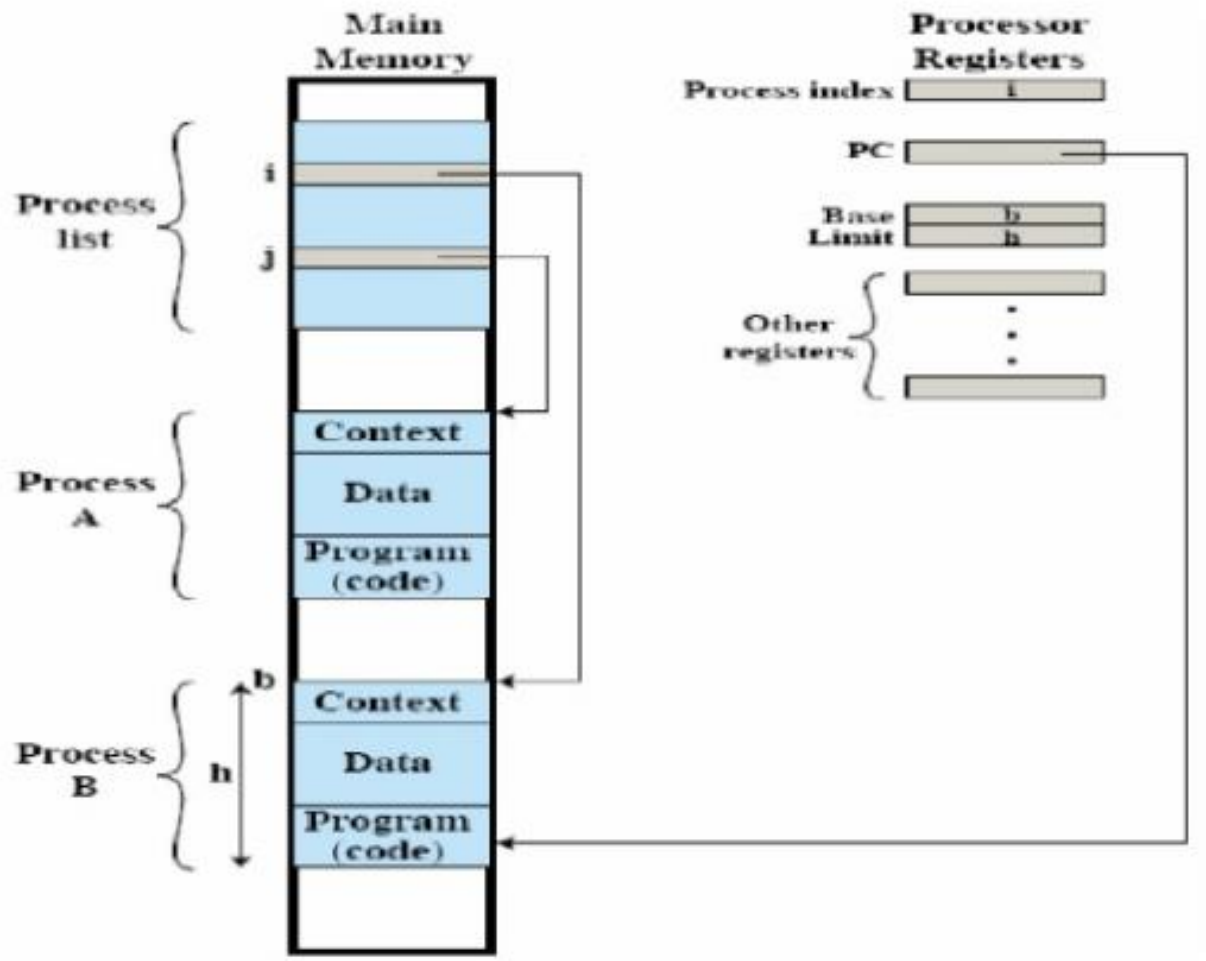
Process Control Block (PCB)

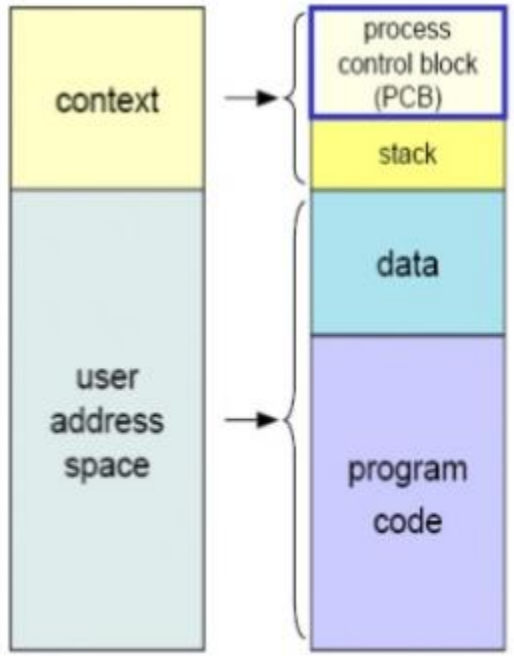
- **PCB içinde**,
- *process numarası,*
- *saklayıcı değerleri,*
- *program sayaç değeri,*
- *process'in ana bellekteki başlangıç adresi (eğer process ana bellekte ise),*
- *process durumu,*
- *öncelik değeri,*
- *process'in oluşturulma zamanı,*
- *kullanılan CPU zamanı,*
- *mesaj kuyruk göstergeci,*
- *statü değerleri,*
- vb bilgiler saklanır.

Process Control Block (PCB)

- Process'lerin bağlam değiştirmeleri (context switching) sırasında;
- *işlemci bağlamı saklanır,*
- *çalışmakta olan process'in PCB'si güncellenir,*
- *çalışmakta olan process uygun kuyruğa (hazır/bloke) alınır,*
- *çalışacak yeni process belirlenir,*
- *seçilen process'in PCB'si güncelleştirilir,*
- *bellek yönetimi ile ilgili bilgiler güncellenir,*
- *seçilen process'in bağlamı (context) yazmaçlara yüklenir ve çalıştırılır.*

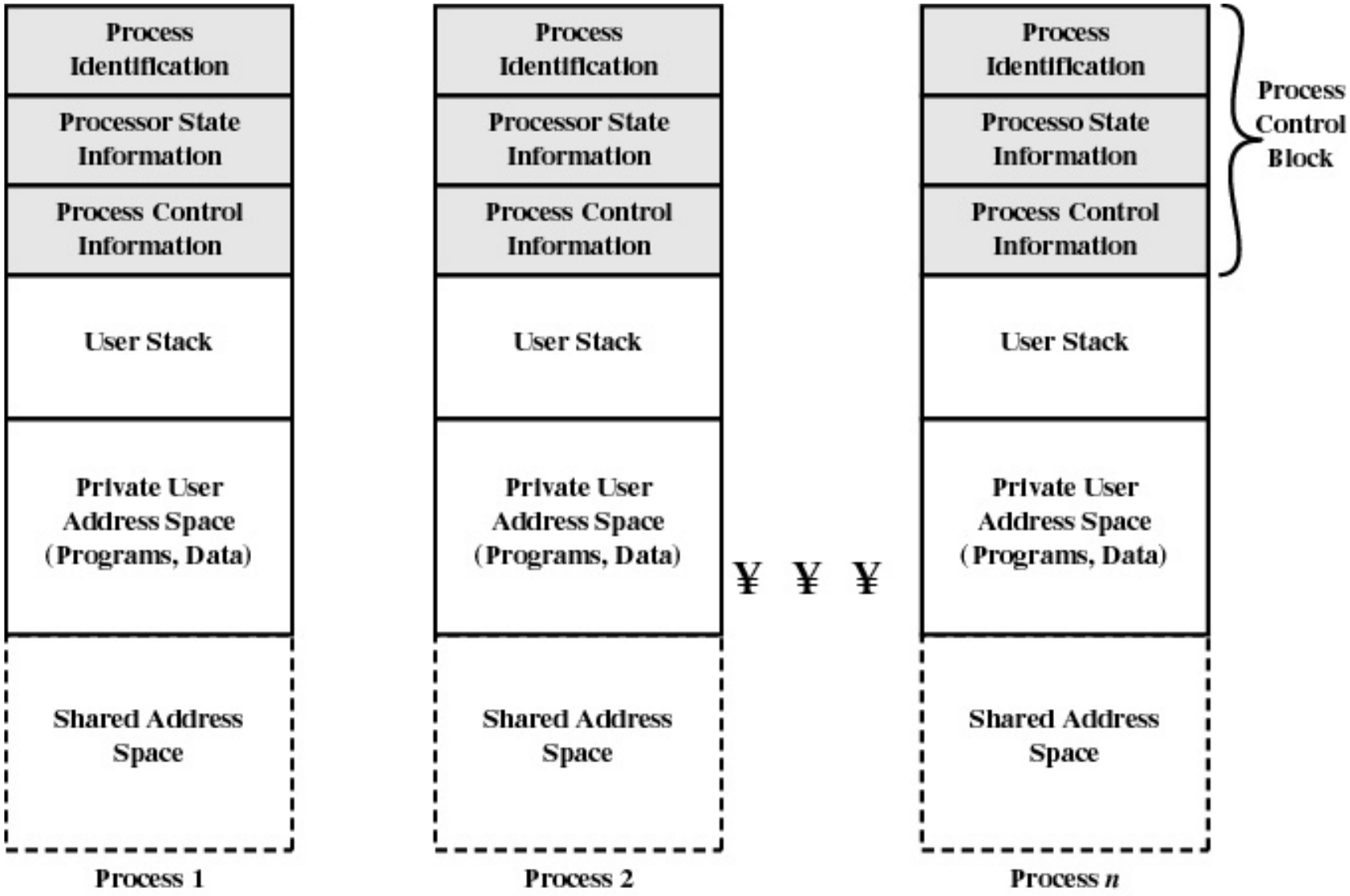
Process Control Block (PCB)



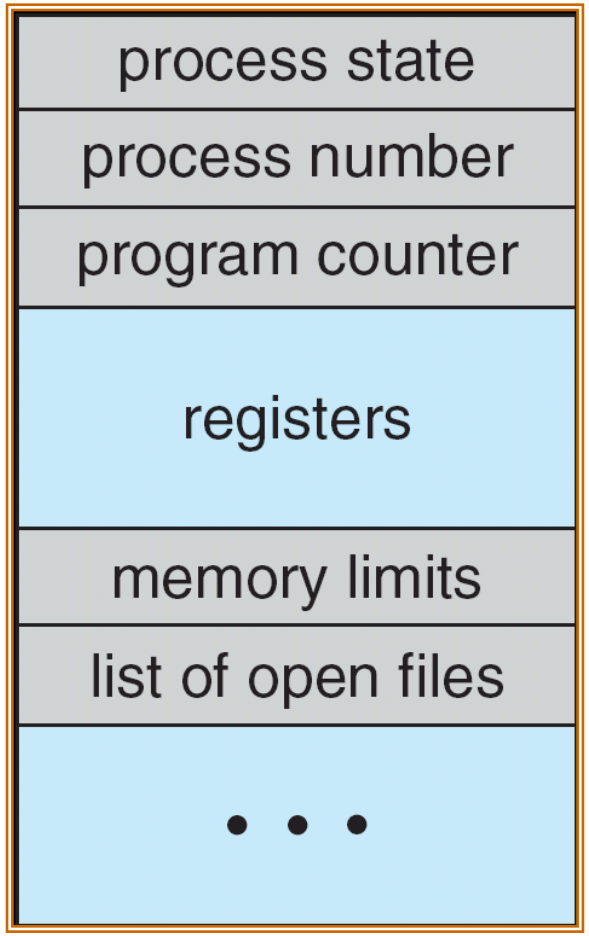


Typical process image implementation

Process Control Block (PCB)



Process Control Block (PCB)



Process Identification

Processor State Information

Process Control Information

Process Control Block (PCB)- Process Table

- **Process Kimlik Bilgileri (Process Identification)**

❑Kimlik Bilgileri

- Process kimlik numarası
- Process’in üst process’inin kimlik numarası
- Üst process’in kullanıcı kimlik bilgisi

Process Control Block (PCB)- Process Table

- **İşlemci Durum Bilgisi (Processor State Information)**

❑Kullanıcıya açık saklayıcılar

- İşlemcinin makina dili kullanılarak erişilebilen saklayıcıları.

❑Kontrol ve Durum saklayıcıları

- Program sayacı
- Durum saklayıcısı
- Yığın işaretçileri
- Program durum sözcüğü (çalışma modu biti var)

Process Control Block (PCB)- Process Table

- **Process Kontrol Bilgileri (Process Control Information)**

❑İş sıralama ve durum bilgileri

- Process durumu
- Önceliği
- İş sıralama ile ilgili bilgiler (Hangi bilgiler olduğu kullanılan iş sıralama algoritmasına bağlı. Örneğin: bekleme süresi, daha önce koştuğu süre)
- Çalışmak için beklediği olay

❑Veri Yapıları

- Process'ler örneğin bir çevrel kuyruk yapısında birbirlerine bağlı olabilir (örneğin aynı kaynağı bekleyen eş öncelikli process'ler).
- Process'ler arasında üst-alt ilişkisi olabilir

Process Control Block (PCB)- Process Table

- ❑Process'ler arası haberleşme ile ilgili bilgiler
- Bazı bayrak, sinyal ve mesajlar process kontrol bloğunda tutulabilir.
- ❑Process Ayrıcalıkları
- Bellek erişimi, kullanılabilecek komutlar ve sistem kaynak ve servislerinin kullanımı ile ilgili haklar
- ❑Bellek yönetimi
- Process'e ayrılmış sanal bellek bölgesinin adresi
- ❑Kaynak kullanımı
- Process'in kullandığı kaynaklar: örneğin açık dosyalar
- Process'in önceki işlemci ve diğer kaynakları kullanımına ilişkin bilgiler